

PaperPass[免费版]查重报告

简明打印版

查重结果(相似度):

总体: 8%

本地库: 8% (本地库包含学术联合库、期刊库、学位库、会议库、共享联合库)

互联网: (免费版不检测互联网资源)

检测版本: 免费版(仅检测中文)

报告编号: R0QC69E76A08B31B6

论文题目: 05

论文作者: 佚名

论文字数: 39119

段落个数: 606

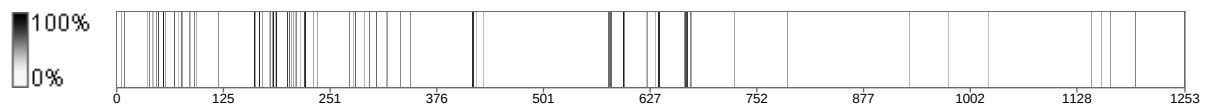
句子个数: 1253

提交时间: 2026-4-21 20:14:00

比对范围: 学术联合库、期刊库、硕博学位库、会议库、共享联合库

查询真伪: <https://www.paperpass.com/check>

句子相似度分布图:



本地库相似资源列表(学术联合库、期刊库、硕博学位库、会议库、共享联合库):

- 相似度: 1.5% 来源: 学术联合库
- 相似度: 1.4% 篇名: 《基于Spring Boot的校园轻博客系统的设计与实现》
来源: 学位

基于大语言模型的简历分析和模拟面试系统

1 绪论

1.1 课题背景

随着数字经济的快速发展,企业招聘规模持续扩大,人力资源部门面临海量简历筛选与面试安排的繁重压力。查重 50%据艾瑞咨询统计,2024 年中国网络招聘市场规模已达 183 亿元,查重 42%网络招聘作为企业最主要招聘渠道占比稳定在七成左右。传统招聘流程中,简历筛选高度依赖 HR 的个人经验与主观判断,不仅效率低下,而且容易因疲劳或认知偏差导致优秀人才被遗漏。与此同时,求职者在面试准备阶段缺乏针对性的练习工具,难以在真实面试中充分展现自身能力。如何在招聘端提升筛选效率与客观性,在求职端提供个性化的面试训练支持,已成为人力资源数字化转型中的重要课题。查重 62%

查重 62%近几年,以大语言模型 (Large Language Model, LLM) 为代表的人工智能技术在自然语言理解和生成方面展现出前所未有的潜力。大语言模型是一类由具有大量参数的人工神经网络构成的语言模型,通常通过自监督学习或半监督学习方式,在大量未标注文本上进行训练,是当前生成式人工智能技术的核心组成部分,在机器翻译、问答系统、对话生成等多个任务中表现出色。与传统监督学习方法相比,大语言模型在少样本甚至零样本条件下即可完成开放域文本理解任务^[1],查重 41%能够从非结构化简历文本中提取关键信息、进行多维度评估并输出条理清晰的反馈。研究表明,"预训练语言模型通过自监督学习在海量无标注文本上获得的语义理解能力,已成为垂直领域智能化改造的核心驱动力"^[2]。进一步的研究总结了大模型技术体系在垂直领域的落地路径,"认为其在招聘、医疗、金融等知识密集型场景中具有显著的应用潜力"^[3]。垂直领域大模型更侧重于满足特定行业的个性化需求,在深入适配用户具体应用场景方面具有显著优势。这些能力恰好对应了招聘场景中对简历自动解析与评分的迫切需求——HR 不再需要逐行阅读海量简历,系统即可快速给出结构化评价;同时也为求职者提供了基于自身简历内容生成个性化面试题、模拟真实对话并获得改进建议的可能。正是基于这一技术契机,将大语言模型引入招聘与求职辅助平台成为兼具现实需求与技术可行性的研究方向。

本课题设计并实现了基于大语言模型的简历分析和模拟面试系统(以下简称"本系统"),定位是帮求职者把面试准备这件事做得更系统一点。架构上选用 B/S 方案,后端主服务用 Spring Boot + Java 21 搭建,语音服务独立采用 Python FastAPI 构建为微服务;前端用 React 18.3 和 Vite。数据层采用 PostgreSQL 并集成 pgvector 扩展,同时承载关系型数据与向量数据的存储,省去单独维护向量数据库的麻烦,Redis Stream 则接手简历

分析、面试评估、知识库向量化这几个耗时任务通过 Redis Stream 异步调度执行

。大模型能力通过 Spring AI 框架接入阿里云 DashScope 的 API，最终落地了四个功能：简历智能解析、个性化模拟面试、实时语音交互、RAG 知识库问答。

1.2 国内外研究现状及发展趋势

1.2.1 大语言模型在简历筛选领域的研究现状

大语言模型在人力资源领域的应用，大致经历了从关键词匹配到语义理解再到 AI 原生评估的几个阶段。早期简历筛选系统的核心是规则引擎，配合 TF-IDF、BM25 等信息检索算法，通过关键词匹配判断候选人与岗位的契合度。这种方式的局限十分明显：难以处理同义词替换、语义隐含和跨领域技能迁移等问题，筛选效果有限。

语义嵌入方向的突破来自 Sentence-BERT。该方法"利用孪生与三元组网络结构生成语义有意义的句子嵌入"，"在保持 BERT 准确率的同时，将相似句对搜索时间从 65 小时缩短至约 5 秒"^[4]，显著提升了语义检索的工程可用性。在招聘垂直场景里，CareerBERT"将孪生网络与多负例排序损失相结合，在共享嵌入空间中表示简历与欧洲职业技能分类职位描述"，"实验表明其在人类专家评估中优于传统与最先进的嵌入方法"^[5]。此外，TAPJFNN 引入主题感知机制，"利用历史招聘数据自动学习岗位需求与求职者经历之间的语义映射"^[6]，在人才搜索和岗位推荐任务上均取得了优于基线的结果。

2024 年前后，围绕大语言模型的简历筛选研究明显增多。应用层面，有研究指出"当前企业招聘面临周期冗长、人岗匹配度不足、人力成本高企等痛点，传统招聘模式已难以适配数字化时代需求，人工智能技术凭借算法优势，正逐步渗透简历筛选、面试评估等核心环节"^[7]。推荐系统方向，有研究"基于协同过滤和深度学习构建了网上招聘智能推荐系统，通过融合用户画像与岗位特征实现精准匹配"^[8]，为在线招聘平台的信息过载问题提供了系统化解方案。POA-Apriori-MR-CNN 混合方法在远程教育推荐任务中取得了较高的推荐准确率（Recall@10=0.8896）^[9]，验证了深度学习在个性化推荐中的有效性。此外，有研究从事业单位人力资源管理视角出发，"提出通过构建精准人才画像与岗位画像、优化选人用人机制来实现人岗智能匹配"，并强调"人才画像需从硬实力与软实力两大维度进行动态优化"^[10]。

随着互联网技术的飞速发展，企业单位在招聘时越来越把目光看向了网络渠道，但是，随着现在网络数据的快速膨胀，数据量正在逐渐以指数增长的态势，使得有限的人力实在无法进行高效准确地处理。这一现实困境进一步凸显了智能简历筛选与推荐技术的必要性，李家鹏在其研究中就探讨了网上招聘智能推荐系统在招聘中的应用，目的是为了提高招聘流程的效率，并为求职者和雇主之间搭建一个更高效、精准的匹配平

台。为实现这一目标，该研究提出了一种基于内容推荐的简历筛选算法，首先通过简历人员的硬性指标进行筛选，接着对简历中的关键信息进行量化处理和权值分配，并通过 Pearson 相似度计算来计算简历与招聘岗位之间的相似度，最后进行分析，为简历筛选的精准化、高效化提供了具体的技术路径，进一步丰富了推荐系统方向在简历筛选领域的研究成果。

此外，LLM 简历筛选的公平性隐患已引起学术界广泛关注，已有研究表明开源语言模型可能对特定人群产生系统性偏见。随着数字技术在就业市场的应用，算法歧视、隐性筛选等新型歧视形式逐渐显现，成为反就业歧视规制的新难点；算法歧视主要表现为用人单位或平台通过算法模型嵌入带有倾向性的筛选规则，部分招聘平台算法甚至根据“性别=女”“户籍=外地”等标签自动降低简历匹配权重。这提醒本系统在后续迭代中不仅需要关注评估效率，还需建立定期的公平性审计与对抗鲁棒性测试机制。

1.2.2 模拟面试与智能问答系统的研究现状

早期的模拟面试平台大多依赖固定题库，无论候选人背景如何，面对的题目都千篇一律，反馈也只是一份静态的评估报告，求职者很难从中获得持续改进的动力。大语言模型的对话生成能力出现之后，这一局面发生了明显改变。有研究提出了基于多智能体协同的大学生就业面试训练系统‘GEMINI+互感评估’工作流，通过多智能体分工协作实现面试题目的自动生成、回答评估与反馈优化，“为模拟面试系统的工程化设计提供了可复用的架构参考”^[11]。本系统后续计划将静态报告升级为可对话的反思交互，这一架构思路是重要的参考之一

从工程实现角度看，当前主流 AI 面试系统通常综合使用自然语言处理、语音识别（ASR）、语音合成（TTS）和计算机视觉等技术。多模态融合分析逐渐成为近年来的研究热点，部分方案尝试将语音语调、文本语义和面部表情结合起来进行三重评估。不过现有系统在处理回答的深层含义、上下文关联和隐喻表达时仍然表现欠佳，情感交互的自然度也还有提升空间。

1.2.3 RAG 技术与知识库问答的研究进展

检索增强生成（Retrieval-Augmented Generation, RAG）技术的核心思路是在大语言模型生成答案之前，先从外部知识库中检索相关文档片段作为上下文输入，以此抑制模型幻觉并提升专业领域回答的准确性。有综述研究总结了 RAG 的实际效果：“通过动态引入外部数据库的信息，RAG 在准确性和可信度上均有明显提升，特别是在需要最新领域知识的场景中”^[12]。进一步的研究综述了多模态检索增强生成（MRAG）在文档问答中的研究进展，“指出传统 RAG 方法在图文混合、长文档及跨文档推理任务中表现受限，

而 MRAG 正由面向静态相似度匹配的检索机制演进为以生成与推理需求为中心的动态证据规划范式"^[13]。在工程实现方面,有研究"设计了基于检索增强生成的私有化 AI 问答系统,利用 Neo4j 与 MySQL 存储企业私有资源,结合语义依存树与检索器实现高精度意图理解和答案生成", "实验表明其归一化折损累积增益值稳定在 0.85 以上"^[14]。此外,孔繁恒等人基于检索与生成混合模型设计并实现了个性化聊天机器人系统,为检索增强生成在对话场景中的落地提供了早期工程参考^[15]。上述研究为本系统知识库问答模块的设计与后续演进提供了重要参考。

1.3 论文组织结构

全文共分六章。

第一章是绪论,从招聘行业当下面临的实际问题切入,说明为什么有必要把大语言模型引入简历分析和模拟面试这两个场景,并梳理国内外相关研究的现状和走向。

第二章做系统需求分析,围绕使用本系统备考面试的求职者展开,把简历管理、模拟面试、实时语音面试和知识库管理四个模块的功能需求逐一细化,同时从性能、可靠性、安全性、兼容性几个角度提出非功能层面的约束条件。

第三章是系统总体设计,给出 B/S 三层架构的整体方案,划清表现层、服务层和数据层各自的职责边界,借助功能结构图、E-R 图和数据流图把核心实体关系和数据流转路径说清楚,附上主要数据表的详细设计。

第四章是系统详细设计与实现,也是全文篇幅最重的一章,按模块逐一展开:设计思路是什么、关键技术为什么这么选、有代表性的代码怎么写、前端页面的交互细节如何处理。

第五章是系统测试与分析,先介绍测试环境的软硬件配置,然后针对四大模块设计并执行功能测试用例,再通过接口响应、并发压力、长文本评估和向量检索四类性能测试,验证系统在实际负载下的稳定性和可用性。

第六章是总结与展望,回顾整个课题做了哪些工作,提炼出三个核心创新点,正视当前实现里存在的问题,最后对后续可能的改进方向做一些探讨。

2 系统需求分析

2.1 系统目标

本系统面向求职者的面试准备场景,旨在利用大语言模型技术提升简历分析的客观性与模拟面试的个性化程度。系统需要达到以下目标:

(1) 简历解析与评估:支持 PDF、DOCX、DOC、TXT 等主流格式的简历文件上传与解析,调用大语言模型生成包含多个评估维度的结构化分析报告^[1],支持导出 PDF 格式

报告。系统应使求职者能够清晰了解简历的优势与改进方向。

(2) 个性化模拟面试：基于候选人上传的简历内容动态生成面试题目，而非从通用题库中随机抽取，支持多轮智能追问，还原真实面试的对话节奏。

(3) 语音交互：集成自动语音识别 (ASR) 与语音合成 (TTS) 能力，支持用户语音输入答题并接收语音反馈，提升面试练习的沉浸感与交互自然度。

(4) 知识库问答：支持用户上传专业文档构建个人知识库，系统基于检索增强生成 (RAG) 技术在文档中检索相关内容后生成回答^[12]，适用于备考知识点明确的岗位。

(5) 稳定性与可扩展性：系统采用 B/S 架构，后端基于 Spring Boot 构建，前后端通过 RESTful API 进行分离^{[16][17]}。后端支持一定并发负载，耗时任务通过异步队列处理，前端适配主流 PC 浏览器，数据库层面具备基本的安全保障。

2.2 系统功能需求

通过对用户场景的分析，本系统面向使用本系统进行面试准备的求职者，按实际使用路径划分出简历管理、模拟面试、实时语音面试和知识库管理四个核心功能模块，以下逐一说明各模块的具体需求。

2.2.1 简历管理模块需求

简历管理是用户进入系统的第一个入口，也是后续模拟面试和知识库功能的数据基础。用户能够上传 PDF、DOCX、DOC 或 TXT 格式的简历文件，系统在上传时校验文件类型与大小，同时通过 SHA-256 内容哈希识别重复文件，避免同一份简历被多次解析和评分。上传成功后，简历文本提取和 AI 评分不应阻塞前端页面，而是借助 Redis Stream 异步投递给消费者处理，状态在 PENDING、PROCESSING、COMPLETED 和 FAILED 之间自动流转。分析完成后，前端需要展示总分以及内容完整性、结构清晰度、技能匹配度、表达专业性、项目经验五个子维度的得分，并给出简历的优点和改进建议。此外，用户还应能一键导出结构化的 PDF 分析报告，并随时查看、排序或删除历史上传记录。

2.2.2 模拟面试模块需求

模拟面试模块的核心目标是让求职者获得贴近自身简历的个性化练习体验。用户选定一份简历后，系统应能基于其中的技术栈和项目经验自动生成一份带参考答案的面试题列表。面试过程中采用逐题作答模式，支持文本输入，开启语音服务后也可通过麦克风语音输入。系统支持智能追问功能：每道主问题可附带若干追问，用户回答完主问题后自动展示追问内容，主答案与追问答案合并存储，形成线性问答流^[11]。所有题目作答完成后，系统调用大语言模型对每道回答进行 0-100 分的评分并给出文字反馈。考虑到用户可能在多道题中给出较长回答，评估环节采用分批评估策略，将题目按批次切分

后分别调用 LLM 评分，再通过二次汇总生成总体评价、优势分析和改进建议，以此规避 Token 溢出问题。最后，用户应能将完整的面试结果导出为 PDF 报告。

2.2.3 实时语音面试模块需求

实时语音面试是模拟面试的增强形态，重点解决交互自然度问题。该模块需要集成 ASR 和 TTS 能力：用户在面试时可以通过麦克风语音输入，系统调用 ASR 服务将音频转为文本后送入答题流程，研究表明结合 Conformer 编码器与 N-gram 语言模型的方案在中文语音识别任务上具有较好的准确率与实时性平衡^[18]；系统生成的问题和反馈则通过 TTS 服务合成为语音播放给用户，多尺度表现力语音合成方法能够在保持自然度的同时对全局韵律和音素级基频进行精细建模^[19]。前端需要维护一套完整的状态机，覆盖 TTS 播报、准备倒计时、录音中、识别中、提交中、完成等阶段，确保语音交互流程连贯顺畅，各状态切换有清晰的 UI 提示。

2.2.4 知识库管理模块需求

知识库管理模块允许用户上传专业文档构建个人知识库，并基于文档内容发起 RAG 智能问答。文档上传支持 PDF、DOCX 和 Markdown 等格式，上传后系统对文档进行解析、分块和异步向量化处理。知识库列表页需要展示文档名称、分类、向量化状态、分块数量等元信息，并支持删除和重新向量化操作。问答环节，用户可以勾选单个或多个知识库作为上下文来源发起提问，系统通过向量相似度检索召回 Top-K 相关片段，结合 LLM 生成回答，并以 SSE（服务器推送事件）流式返回，实现文字逐字出现的打字机效果。为便于长期知识管理，系统还应支持多会话管理，用户可以创建新的问答会话，查看历史消息，并对会话进行置顶或归档。

2.2.5 非功能需求

除功能需求外，系统还需满足以下几类非功能约束。**在系统工程及需求工程中，非功能性需求是在一定条件下系统的运行状况以及系统特性需要达到某些普遍或特定的要求，****一般包括执行质量和发展质量两类：执行质量是指在系统运行时所具备的特性，如易用性；发展质量是指在开发过程中软件系统结构所具备的特性，如可扩展性、可测试性等。****据此，本系统的非功能需求从执行质量与发展质量两个维度出发，主要涵盖性能、可靠性、安全性与兼容性四个方面。**

在性能方面，简历解析、面试评估和知识库向量化等耗时操作必须采用异步处理，前端通过轮询实时获取任务进度。对于普通查询接口的响应时间，本系统要求控制在 500ms 以内，以确保用户获得即时反馈。同时，参照业界通用的 2/5/10 响应时间原则：**当用户在 2 秒以内得到响应，则认为系统的响应速度很快；当用户在 2-5 秒之间得到响**

应, 则认为系统的响应速度可以接受; 当用户在 5-10 秒以内得到响应, 则认为系统的响应速度很慢, 但是仍在可接受的范围内; 当用户在超过 10 秒后仍然无法得到响应时, 大多数用户则认为当前请求失败, 并认为系统极差。本系统通过异步机制将耗时任务的总体反馈时间控制在 10 秒以内, 普通查询接口则严格遵循 500ms 的极速响应标准, 确保核心交互处于"响应速度很快"的可接受区间。

在可靠性方面, 异步任务需要具备失败自动重试能力, 上限为 3 次; 数据库操作通过 JPA 统一事务管理, 防止数据不一致。

在安全性方面, 用户上传的原始文件应存放于 S3 兼容的对象存储中, 数据库仅保存文件元数据; 对于上传、创建面试等敏感接口, 需要实施 API 限流保护。

在兼容性方面, 前端采用响应式布局适配主流 PC 浏览器; 语音服务作为可选模块独立部署, 即使语音服务不可用, 也不应影响简历管理和文本模拟面试的核心功能。

2.3 本章小结

本章首先明确了系统面向求职者的建设目标, 随后围绕简历管理、模拟面试、实时语音面试和知识库管理四个模块逐条梳理了功能需求, 并将文本/语音输入、异步处理、RAG 问答、多会话管理等关键能力细化到具体场景中。最后从性能、可靠性、安全性与兼容性四个维度补充了非功能需求约束。上述内容将作为后续系统总体设计和详细实现的重要输入。

3 系统总体设计

3.1.1 系统整体架构

系统采用 B/S 架构, 用户用浏览器就能用全部功能, 无需安装额外的客户端。有研究指出: "B/S 体系结构由浏览器端 (Browser) 和服务器端 (Server) 组成, 并且是基于 C/S 体系结构发展而来的网络结构模型。在这种结构下, 客户端统一使用浏览器来实现, 并且浏览器端仅处理很少的业务逻辑, 主要负责数据的传送和显示, 绝大多数的业务逻辑将由服务器端来具体实现。这样的结构使得用户不必再负载多余的客户端程序, 开发人员和维护人员的工作量以及相应成本也就随之降低, 进而降低了总成本。"

整体纵向分成数据层、服务层和表现层三个层次。正如研究所述: "三层架构是分层结构中很常用的一种结构, 是在用户访问端与数据访问端之间插入一个中间层, 从而分为表示层、业务逻辑层以及数据访问层。"本系统中, 层与层之间通过 RESTful API、JDBC、S3 SDK 这类标准接口进行交互。这种分法的好处是: 哪层要换技术栈, 只要接口约定不动, 对其它层的影响就能控制在最小范围内。系统整体架构如图 1 所示

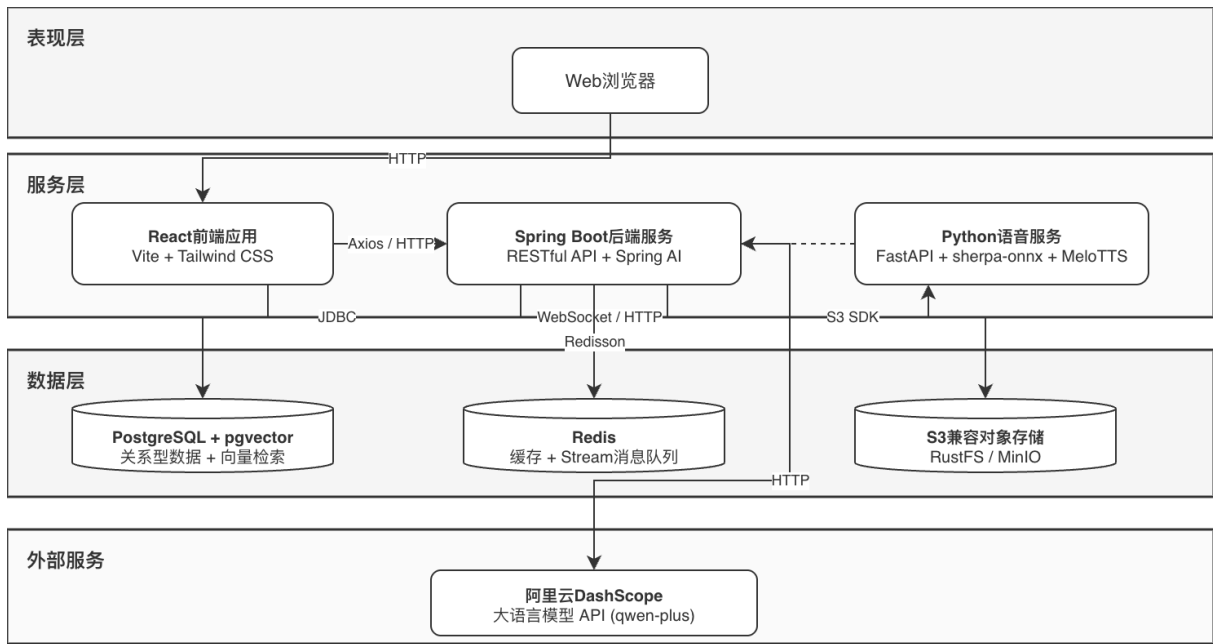


图 1 系统架构图

(1) 表现层：直接面向用户，负责界面渲染和交互响应。前端基于 React 18.3 和 TypeScript 开发。传统动态网站需频繁操作 DOM 树节点以更新界面，对于复杂页面，频繁的 DOM 操作易引发性能瓶颈并增加开发复杂度；React 通过实现 VisualDom 机制，以组件化思想为前端开发提供了新的思路，其核心算法 ReactDiff 通过制定策略将 $O(n^3)$ 复杂度转换为 $O(n)$ ，确保仅对界面中需更新的部分进行差异化刷新，有效规避了页面整体重绘引发的渲染性能损耗，使开发者能够聚焦于业务逻辑实现。构建工具采用 Vite，依托浏览器原生 ES 模块实现按需编译，显著提升了开发服务器的冷启动速度，并具备良好的大规模项目构建性能。样式方案采用 Tailwind CSS，通过原子化类名直接内联于组件标记中，降低了样式文件与组件模板间上下文切换的开发成本。前后端数据交互主要采用 Axios 发起 HTTP 请求；针对知识库问答场景，则基于 EventSource 消费 SSE 流，以此实现服务器推送的流式文本逐字呈现效果。

(2) 服务层：所有业务逻辑和外部服务调用均在此层处理。服务层拆分为两个独立部署的单元：主服务基于 Spring Boot 与 Java 21 构建。Spring Boot 作为 Java EE 领域广泛应用的轻量级框架，继承了 Spring 框架的优秀基因，专为简化项目配置而生，能够轻松创建独立的、生产级的、可直接运行的应用程序。其核心理念在于约定优于配置，通过减少开发过程中需要决策的数量，在获得简洁性的同时保持灵活性；开箱即用的设计策略使得项目仅需在构建配置文件中添加少量依赖，即可集成大量易于使用的基础框架，将原本繁杂的配置工作简化到近乎零代码、零 XML 的程度，依赖包的版本兼容性问题也

由框架统一解决。Spring AI 框架负责统一对接大模型 API，Spring Boot 的自动配置与内嵌服务器消除了大量模板化配置，显著提升了服务的构建与部署效率[16]。语音服务则独立采用 Python FastAPI 框架构建，内部集成 sherpa-onnx 实现中文语音识别、MeloTTS 实现中文语音合成——鉴于语音模型与 Python 生态耦合度较高，若强行整合至 Java 主服务将引入不必要的复杂性与维护负担，故将其拆分为独立微服务，**前端与主服务均通过标准化 HTTP 接口进行调用**。胡荣等人的研究亦表明，基于 Spring Boot 的前后端分离架构对开发效率与可维护性均有显著促进作用^[17]。

(3) 数据层：全量数据的持久化和检索均由数据层负责。**数据库选用 PostgreSQL。**互联网技术的发展产生了海量非结构化数据，这类数据在传统关系型数据库中难以被高速有效地进行存储和处理；各类 NoSQL 数据库虽能有效存储处理非结构化数据，但对关系运算功能的弱化难以满足应用场景的需求。而具备非结构化数据处理能力的新型关系型数据库则提供了适用多种应用场景的高效存储方式。PostgreSQL 拥有 XML、JSON 和 hstore 三种存储非结构化数据的方法，**对于带有部分简单非结构化数据且需要进行复杂关系运算操作的应用，建议使用关系数据模型加 hstore 数据类型结合的方法进行存储。**"在此基础上，通过安装 pgvector 扩展，PostgreSQL 不仅支持关系型数据存储，亦支持向量数据存储与相似度检索，RAG 问答所需的高维向量查询由 pgvector 承担。陈建海等人对微服务架构 B/S 系统的研究表明，合理的服务拆分配上负载均衡，并发处理能力有很大的提升^[20]。Redis 在这里同时承担两项职能：一是缓存面试会话状态，比本地 ConcurrentHashMap 的好处是多实例部署时状态能共享；二是当消息队列用，基于 Redis Stream 调度简历解析、知识库向量化、面试报告生成这几个不能让用户等的耗时任务。用户上传的简历文件和知识库文档这类二进制数据单独放 S3 兼容对象存储（RustFS/MinIO），**数据库仅保存元数据与访问路径，实现结构化数据与二进制文件对象的解耦存储，降低耦合度并便于独立扩展与权限管控。**

(4) 外部服务层：通过 Spring AI 框架接入阿里云 DashScope 的大模型 API 默认采用 qwen-plus 执行简历分析、面试题生成等任务。API 密钥管理、超时控制和异常重试都在后端统一封装，不暴露给前端。

3.1.2 系统功能结构

系统按需求分析结果划分为四个核心功能模块：简历管理、模拟面试、实时语音面试和知识库管理，另外还有上传组件、历史记录管理、评分可视化这些公共功能。系统功能结构如图 2 所示。

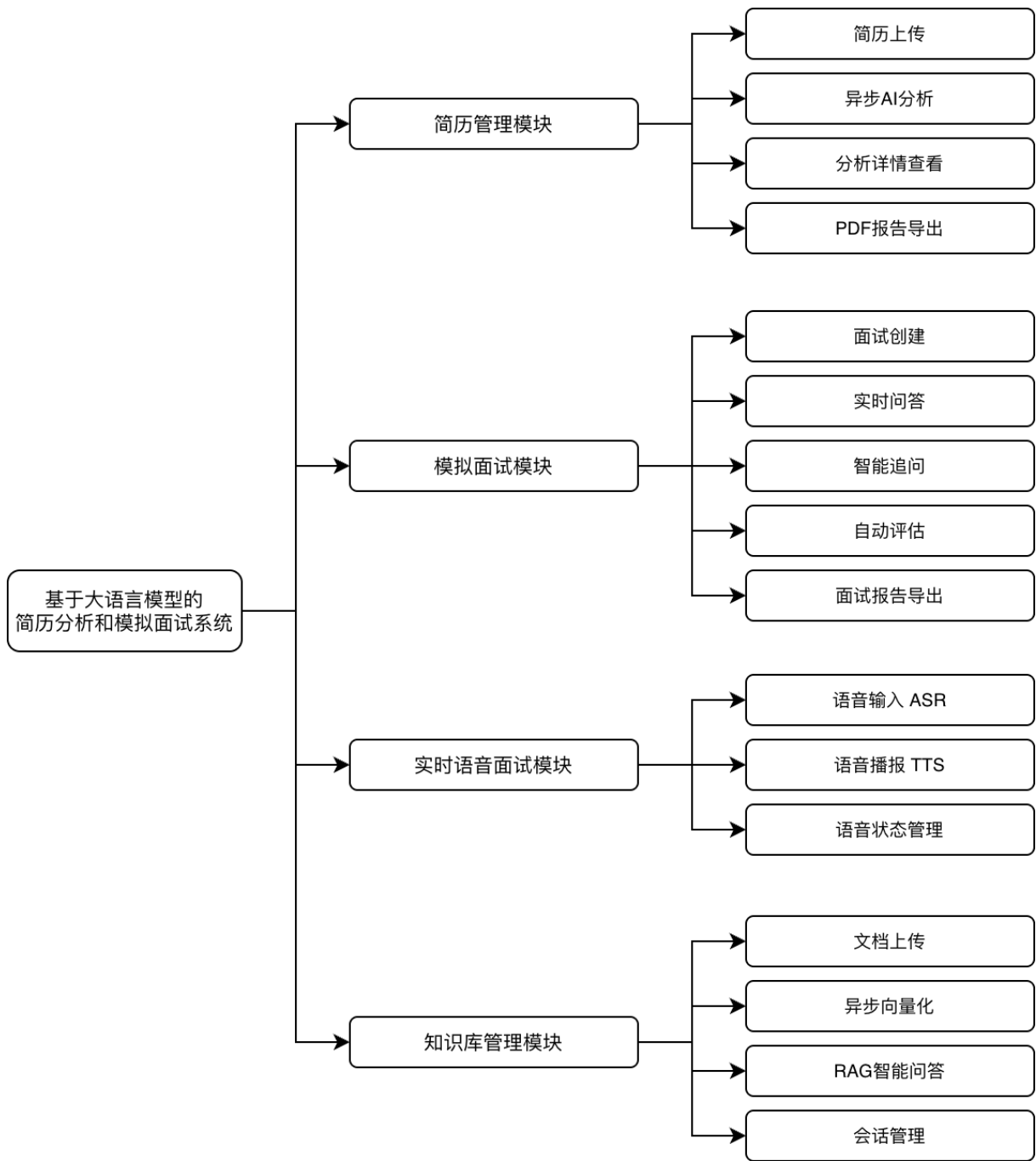


图 1 系统架构图

各模块说明如下：

- (1) 简历管理模块：处理简历上传、状态查看、AI 分析结果展示和 PDF 导出。用户上传简历后，Apache Tika 负责提取文档文本，经 Redis Stream 异步交由 LLM 分析，最终生成内容完整性、结构清晰度、技能匹配度、表达专业性、项目经验五个维度的评分和改进建议。
- (2) 模拟面试模块：用户选好简历，系统根据简历内容生成个性化题目，不从通

用题库随机抽。面试过程逐题推进，支持配置多轮追问，构建线性问答流。答完后对每道题评分反馈，汇总成总体评价并支持导出 PDF。

(3) 实时语音面试模块：模拟面试的语音增强版，将文本输入替换为语音交互。集成 ASR 和 TTS, 用户语音输入实时转文字, 面试问题和评估反馈也能以语音形式播报, 交互自然度优于纯文字模式。

(4) 知识库管理模块：用户上传专业文档建个人知识库，系统解析分块后异步向量化存进 pgvector。用户发问时，系统做向量相似度检索召回相关片段，LLM 基于这些片段生成回答，SSE 流式推给前端。

3.2 系统数据分析

3.2.1 实体关系分析

整理业务需求后，系统识别出七个核心实体：简历（Resume）、简历分析结果（ResumeAnalysis）、面试会话（InterviewSession）、面试答案（InterviewAnswer）、知识库文档（KnowledgeBase）、RAG 聊天会话（RagChatSession）、RAG 聊天消息（RagChatMessage）。各实体属性和相互关系如图 3 所示

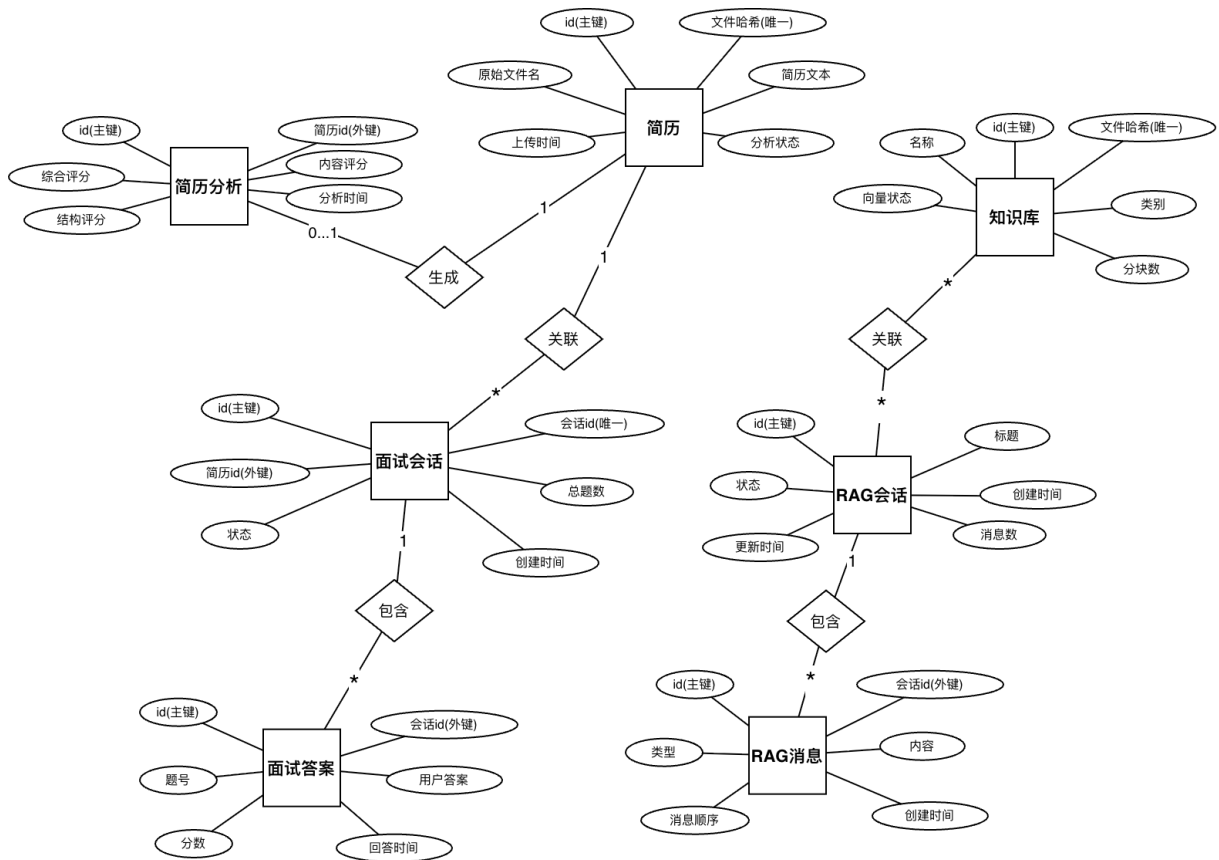


图 3 系统整体 E-R 图

各实体关系说明如下：

(1) 简历与简历分析结果：一份简历分析完生成一条分析记录，一对一关系。ResumeEntity 用 analyzeStatus 字段跟踪分析状态，ResumeAnalysisEntity 通过 resume_id 外键关联。

(2) 简历与面试会话：一份简历可以用来发起多次模拟面试，一对多关系。InterviewSessionEntity 通过 resume_id 外键关联 ResumeEntity。

(3) 面试会话与面试答案：一个面试会话包含多道题目和对应的用户回答，一对多关系。InterviewAnswerEntity 通过 session_id 外键关联 InterviewSessionEntity，questionIndex 和 session_id 的组合唯一约束保证每道题只有一条「方案选单」回答记录。

(4) RAG 聊天会话与 RAG 聊天消息：一个会话包含多条来回消息，一对多关系。RagChatMessageEntity 通过 session_id 外键关联 RagChatSessionEntity，messageOrder 字段保证消息顺序。

(5) RAG 聊天会话与知识库文档：一个会话可以关联多个知识库，一个知识库也可以被多个会话引用，多对多关系，通过中间表 rag_session_knowledge_bases 关联。

3.2.2 数据流图

系统顶层数据流图如图 4 所示，外部实体是使用系统备考面试的求职者，数据流主要有简历文件、面试数据、知识库文档和问答消息这几类。

将系统内部展开，得到 0 层数据流图，如图 5 所示。

各模块数据流说明如下：

(1) 简历管理模块：用户上传简历后，文件元数据写 resumes 表，文件本身传 S3。系统往 Redis Stream 发一条异步分析消息，消费者调用 LLM 完成解析和评分，结果写 resume_analyses 表，同时更新 resumes 表的状态字段。

(2) 模拟面试模块：创建面试后，配置和题目列表写 interview_sessions 表。用户每答一题，回答写入 interview_answers 表。面试结束后往 Redis Stream 发异步评估消息，消费者对每道回答调 LLM 评分，最终把总体评价和报告数据更新回 interview_sessions 表。

(3) 实时语音面试模块：用户在前端对着麦克风说话，前端调 Python 语音服务的 ASR 接口拿到识别文本，文本送入模拟面试模块的答题流程。系统生成的回复文本通过 TTS 接口合成语音流，返回前端播放。

(4) 知识库管理模块：用户上传文档后，元数据写 knowledge_bases 表，文件传 S3。系统往 Redis Stream 发异步向量化消息，消费者把文档分块后调 Embedding 模型生

成向量,存进pgvector的vector_store表。用户发起RAG问答时,先把问题向量化,pgvector检索出相关片段,调用LLM生成回答,并通过SSE流式返回至前端。

3.3 数据库设计

查重 64%

数据库选 PostgreSQL,装pgvector扩展支持向量存储。表设计遵循第三范式,在查询频繁字段上建了索引。以下是主要数据表的说明。

3.3.1 简历相关表

resumes 表存用户上传的简历文件元数据,如表 1 所示。

resume_analyses 表用于存储简历 AI 分析结果,如表 2 所示。

interview_sessions 表用于存储模拟面试会话信息,如表 3 所示。

interview_answers 表用于存储面试中每道题的用户回答与评估结果,如表 4 所示。

查重 69%

knowledge_bases 表用于存储知识库文档元数据,如表 5 所示。

rag_chat_sessions 表用于存储 RAG 问答会话,如表 6 所示。

rag_chat_messages 表用于存储 RAG 问答消息,如表 7 所示。

rag_session_knowledge_bases 表是 RAG 会话与知识库的多对多关联中间表,session_id 和 knowledge_base_id 两个外键共同构成复合主键。

另外, Spring AI 框架第一次启动时会自动建 vector_store 表,用来存文档分块后的向量数据。表里有 id、content、metadata、embedding 四个字段,embedding 字段是 pgvector 的 vector 类型,支持余弦相似度检索。

3.4 本章小结

本章完成了系统的总体设计工作。首先说明了 B/S 三层架构中表现层、服务层、数据层和外部服务层的职责边界和主要技术选型理由;随后通过功能结构图展示了四大核心模块的拆分方式;接着利用 E-R 图和数据流图梳理了简历、面试会话、知识库文档、RAG 聊天会话等核心实体之间的关系,以及数据在各模块间的流转路径;最后给出了主要数据表的字段设计和索引策略。以上内容将作为第四章详细设计与实现的基础输入。

4 系统详细设计与实现

本章逐一展开简历管理、模拟面试、实时语音面试和知识库管理四个模块的详细设计与实现,以设计思路和技术决策为主线,配合必要的流转说明和代表性代码片段,不堆砌大段源码。

4.1 简历管理模块详细设计与实现

4.1.1 模块架构设计

简历管理模块是用户进入系统的第一个触点,主要处理简历上传、格式解析、重复

检测、AI 多维评分和 PDF 报告导出。在简历解析技术方面，已有研究"通过规则匹配与统计学习相结合的方法完成了半结构化简历的自动解析"^[21]；有研究"针对简历文本的领域特点对通用 NER 模型进行了适配优化"^[22]，解决了中文简历命名实体识别问题；有研究结合 BERT 预训练模型"对简历关键信息进行了抽取与识别"^[23]；有研究"将 CNN 与多头注意力机制融合应用于简历信息提取，通过局部特征与全局语义的协同建模实现了更高精度的结构化解析"^[24]；此外，有研究从企业大规模招聘场景出发，"研究了基于机器学习的简历自动分类筛选方法，为面向海量简历数据的智能初筛提供了可落地的模型方案"^[25]。为了支撑后续模块对简历数据的高效访问，该模块在实现上采用经典的分层架构，从上至下划分为 Controller 层、Service 层、Persistence 层和异步消息处理层，如图 6 所示。

Controller 层由 ResumeController 统一暴露 RESTful 接口，覆盖上传、分页查询、详情查看、重新分析、PDF 导出和删除。Service 层是业务逻辑的核心：ResumeUploadService 编排整个上传流程，ResumeParseService 靠 Apache Tika 完成多格式文档解析，ResumeStorageService 封装 S3 交互，ResumeQueryService 组装详情和列表数据，ResumeExportService 基于 iText 8 生成 PDF 分析报告。Persistence 层通过 Spring Data JPA 和 PostgreSQL 交互，完成实体持久化和状态更新。异步处理层基于 Redis Stream，AnalyzeStreamProducer 投递分析任务，AnalyzeStreamConsumer 消费任务后调 ResumeGradingService 完成 LLM 评分。

这种分层的核心思路是"对接口编程"和"关注点分离"：Controller 仅负责请求转发和参数校验，Service 专注业务流程，Persistence 屏蔽数据库差异，异步处理层把耗时的大模型调用和主请求线程彻底解耦。这样上传接口能在毫秒级响应，不会因为 LLM 推理延迟拖垮前端。

4.1.2 简历上传与去重流程

简历上传是模块的第一个关键流程，涉及文件校验、内容去重、文本解析、对象存储和数据库持久化五步。系统支持 PDF、DOCX、DOC 和 TXT，大小上限 10MB。用户误操作或网络抖动时很容易重复上传同一份简历，为了不白白消耗存储资源和 LLM 调用费用，系统设计了基于内容哈希的去重机制。

上传接口上，ResumeController 加了自定义的@RateLimit 注解限流，基于客户端 ID 限制每 10 秒最多 1 次上传请求。底层基于 Guava 的 RateLimiter，能防止恶意高频上传把存储和解析资源耗光。

ResumeUploadService.upload()是上传流程的编排核心：先校验文件类型和大小；再

读取全部字节算 SHA-256 摘要，查询 resumes 表以检测是否存在重复记录：若存在则直接返回已有记录；若不存在则继续提取文本、上传至 S3 并持久化至数据库；没有的话才继续提取文本、上传 S3、落库，最后往 Redis Stream 投一条异步分析任务。基于内容哈希去重比比文件名或文件大小可靠多了——用户改了文件名，哈希值还是一样，系统照样能认出来。

简历文本提取由 ResumeParseService 负责，底层是 Apache Tika。Tika 能从 PDF、Office 文档、HTML 等一千多种格式里提取纯文本，通过单一接口完成解析，项目里主要用它的多格式文本提取能力：PDF 走 PDFBox，DOCX 走 Apache POI。上传的 MultipartFile 转成输入流送入 Tika，提取出的原始文本还要经过 cleanText() 后处理，包括合并连续空行、去掉页眉页脚重复文本、替换特殊 Unicode 空白符、过滤不可见控制字符。这几步能明显提升后续 LLM 解析的准确性，减少格式噪声带来的误判。

存储上，文件实体不进数据库，通过 ResumeStorageService 上传到 S3 兼容对象存储（RustFS/MinIO），数据库只存 storageKey 和 storageUrl。这种分离设计减轻了数据库的存储和备份压力，后续要换存储后端只改 storageUrl 前缀配置就行，不动业务代码。

4.1.3 异步 AI 分析流程

简历上传完成后，系统要调 LLM 对简历文本做多维度评分^[1]。LLM 调用通常耗时 1-5 秒，不应阻塞用户界面，所以用 Redis Stream 做异步任务队列，把 AI 分析和主请求线程彻底解耦。

生产端，AnalyzeStreamProducer 把分析任务投到 Redis Stream，消息体包含三个字段：简历 ID (resumeld)、简历文本 (resumeText) 和重试计数器 (retryCount, 初始 0)。投递成功后 upload() 立刻返回，前端开始轮询 analyzeStatus 字段的变化。

消费端，AnalyzeStreamConsumer 继承自抽象的 AbstractStreamConsumer，封装了自动 ACK、死信队列和异常重试。核心流程是：解析出 resumeld 和 resumeText，把分析状态改成 PROCESSING；调 ResumeGradingService.gradeResume() 执行 LLM 评分；评分完把结果存 resume_analyses 表，状态改成 COMPLETED。内置最多 3 次自动重试——网络超时、API 限流或异常返回时，任务重新投回 Redis Stream 并把 retryCount 加 1；超过 3 次就标 FAILED，在 analyzeError 字段记下异常信息方便排查。和传统定时任务轮询比，Redis Stream 有消息持久化、消费者组负载均衡和自动确认这些优势，更适合高并发场景。

ResumeGradingService 基于 Spring AI 的 ChatClient 和阿里云 DashScope 交互。大模型这块的技术基础——从 Vaswani 等人的 Transformer 自注意力机制，到 Devlin 等人的 BERT 双向预训练，再到 Brown 等人证明大模型少样本学习能力——这些工作共同奠定了

本系统简历语义理解能力的基础，也降低了评估维度设计中的标注成本。Spring AI 提供了一套用于和 AI 聊天模型通信的流畅 API，通过对 Chat、Embedding、Vector Store 等能力的抽象实现跨模型可移植性。本模块用 ChatClient 发结构化 Prompt 并接收 JSON 格式的评分结果，Prompt 明确了五个维度和分值：内容完整性（0-25 分）、结构清晰度（0-20 分）、技能匹配度（0-25 分）、表达专业性（0-15 分）、项目经验（0-15 分），同时要求模型给出简历摘要、优点列表和改进建议。明确指定输出格式和评分细则能有效降低模型输出的不确定性，提高结果的可解析性。评分结果出来后，用 Jackson 把 JSON 字符串映射成 ResumeAnalysisResult 对象；解析失败则抛业务异常，由上层重试机制捕获并重新投递。

4.1.4 前端页面设计与实现

简历管理模块前端包含三个核心页面：简历上传页（UploadPage.tsx）、简历库列表页（HistoryPage.tsx）和简历分析详情页（ResumeDetailPage.tsx）。

图 10 展示了简历上传页面的实际效果，用户可通过拖拽或点击方式选择简历文件进行上传。

上传页用拖拽组件 UploadZone，底层基于 HTML5 的 Drag and Drop API 和 `<input type="file">`。用户选完文件，前端先校验类型和大小，再用 Axios 发 multipart/form-data 请求到 `/api/resumes/upload`。上传成功拿到简历 ID 后，React Router 直接跳到分析详情页，同时启动定时轮询。

ResumeDetailPage 展示简历元信息和 AI 分析结果。页面加载时用 useEffect 发详情查询，analyzeStatus 是 PENDING 或 PROCESSING 就起一个 2 秒间隔的定时器轮询，直到变成 COMPLETED 或 FAILED。分析完成后，用 Recharts 渲染雷达图直观展示五个维度得分，紫色主题填充加极坐标网格线，看起来比较清晰。

图 11 展示了简历分析详情页，系统以雷达图形式展示内容完整性、结构清晰度、技能匹配度、表达专业性和项目经验五个维度的评分结果。详情页底部有“导出 PDF 报告”按钮，点了调 `/api/resumes/{id}/export-pdf` 接口，后端用 iText 8.0.5 生成包含基本信息、雷达图、各维度得分、优点列表和改进建议的 PDF，前端收到二进制流后用 `URL.createObjectURL` 创建临时下载链接触发浏览器下载。

4.2 模拟面试模块详细设计与实现

4.2.1 模块架构设计

模拟面试模块承载系统的核心业务流程，目标是基于用户上传的简历生成个性化题目，作答完成后提供自动评分与反馈。答题过程涉及大量短交互和频繁状态变更，所以

在常规分层架构之外额外引入了 Redis 缓存层,降低数据库压力,整体架构如图 7 所示。

InterviewController 暴露 RESTful 接口,包括创建会话、获取当前题目、提交答案、完成面试、获取报告和导出 PDF。InterviewSessionService 是核心服务,负责会话生命周期管理、题目生成和答案提交。AnswerEvaluationService 调用 LLM 对答案进行评分,底层用分批评估策略规避长文本场景的 Token 溢出。Redis 缓存当前会话的完整状态(题目列表、当前索引、用户回答等),避免每次交互均查询数据库。

引入 Redis 缓存的原因很直接:每次提交答案,后端都要更新题号、判断是不是最后一题、可能触发追问。全走 PostgreSQL 的话,连接池压力大,事务锁竞争也会拖慢响应。会话状态缓存到 Redis 后,答题提交的响应时间从几十毫秒降至数毫秒,用户体验显著提升。

4.2.2 面试会话管理与题目生成

杨真等人基于人工智能与虚拟现实技术构建了招聘面试人工智能系统的基本框架,该框架涵盖在线简历填写、智能筛选简历、智能在线聊天面试、场景面试、综合评判与精准录用六个环节,其核心模块包括简历自动筛选、智能机器人聊天面试及 VR 场景面试。在智能聊天面试环节,系统通过网页或移动端应用支持人机交互,使通过简历筛选的候选人可在规定时间内的任意时刻、任意地点完成在线面试。面试聊天机器人运用机器学习与自然语言处理技术,采用交互式问答模式,能够根据对话场景主动向求职者提问,以明晰其实际需求与能力特征;同时,机器人可针对不同候选人做出差异化应答,并基于岗位要求提供信息反馈与后续招聘建议。这种由系统主导、具备实时互动能力的对话式面试模式,为智能化招聘系统的研发提供了重要的理论依据与实践参考。

用户选好简历后,前端调 POST /api/interview/sessions 创建面试会话。InterviewSessionService.createSession()的输入 InterviewConfigRequest 包含题目数量、难度等级、是否启用追问这几个配置项。系统首先查询简历文本与分析摘要,随后将二者融合后传递至 InterviewQuestionGenerator 生成题目列表。

InterviewQuestionGenerator 基于 Spring AI 的 ChatClient 调 LLM, Prompt 里融入了简历内容和分析结果,旨在生成高度个性化的题目^[11]。Prompt 要求输出 JSON 数组,每个元素包含 question、category (如"项目经验"、"技术基础"、"职业规划") 和 followUps (追问列表)。固定输出格式结合追问数量约束,显著提升 LLM 返回结果的稳定性与可解析性。

题目生成完还会调 generateReferenceAnswers()基于简历文本为每道题生成参考答案,评估阶段会把参考答案一起给 LLM,帮助模型更准确地判断用户回答的完整性和深

度。会话实体构建完成后，系统生成全局唯一的 sessionId (UUID)，数据同时写入 PostgreSQL 与 Redis，Redis 缓存键为 interview:session:{sessionId}，过期时间 2 小时。双写策略保证了持久化安全性和高频读取的低延迟。

用户答完提交，调 POST /api/interview/sessions/{sessionId}/answers。InterviewSessionService.submitAnswer() 先从 Redis 读取会话缓存，若缓存未命中则从 PostgreSQL 加载并重建缓存。然后把当前答案存 interview_answers 表，更新 Redis 里的题号索引，判断是不是最后一题。若为最后一题，则将状态置为 COMPLETED 并向 Redis Stream 投递评估任务；否则返回下一题索引，前端加载新题清空输入框。

4.2.3 智能追问与答题评估

每道主问题生成时已经带了一条追问。前端展示主问题后，若用户开了“启用追问”，用户答完主问题就自动展示追问内容。用户答追问提交后，前端把主问题答案和追问答案合并成一条完整答案，走同一个提交接口。这种线性追问流的好处是状态管理简单：每道题对应一条 InterviewAnswer 记录，不用维护复杂的对话树，但依然能模拟面试官针对某个回答深挖的场景。

评估是整个模块里最费 LLM Token 的环节。题目多 (10-15 道) 或者每道回答很长 (几百字) 时，把所有题目和答案一次性拼给 LLM，上下文很容易超限，API 直接报 Input length exceeds maximum context length^[2]。AnswerEvaluationService 针对这个问题设计了分批评估策略。

思路是：按固定批次大小 (默认 8 道) 切分题目，每批分别调 LLM 评估，得到每道题的得分、反馈和关键点；全部批次跑完再做一次二次汇总调用，生成总体评价、优势分析和改进建议。buildBatchEvaluationPrompt() 为每个批次单独构造 Prompt，要求模型按 JSON 数组输出每道题的 questionIndex、score、feedback 和 keyPoints。这套策略有两个实际好处：一是彻底解决了 Token 溢出的问题，测试表明 15 道长文本面试的评估成功率从 0% 提升到了 100%；二是单次 LLM 调用的延迟降低了，短文本生成快，某一批次失败也只需重试那一批，不影响其他题目。

分批评估完成后，summarizeBatchResults() 把所有题目的评分结果、类别和反馈拼成新 Prompt，再调一次 LLM，输出包含 overallScore、overallFeedback、strengths 和 improvements 的 JSON 对象。二次汇总让最终报告能从整体视角提炼核心优势和待改进方向，而不只是罗列平均分，可读性好很多。

4.2.4 前端面试交互设计

InterviewPage.tsx 是模拟面试主页面，用 React Hooks 管理面试状态机，支持配置模

式 (config)、普通答题模式 (interview) 和实时语音答题模式 (realtime)。点"开始面试"后, 前端先调创建会话接口, 成功后根据用户选的模式切换到 interview 或 realtime。

interview 模式下, 顶部进度条宽度按 $(currentIndex + 1) / totalQuestions$ 计算, CSS 过渡动画实现平滑增长。中间展示当前题目和可选追问, 底部是文本输入区和提交按钮。

图 12 展示了模拟面试的答题界面, 顶部显示面试进度, 中间展示当前题目内容, 底部提供文本输入区域供用户作答。用户提交后, isLastQuestion 为 true 就跳报告页, 否则加载下一题清空输入框, 更新进度条。

面试完成后进 InterviewReportPage, 展示各题得分柱状图、面试总分、总体反馈、优势分析和改进建议。

图 13 展示了面试结束后的评估报告页面, 包含各题得分的柱状图、总体评分、优势分析与改进建议等核心内容。若评估状态为 EVALUATING, 页面显示加载动画并每 2 秒轮询一次进度。评估完成后用户可以导出 PDF, 后端生成完整评估报告, 前端触发下载。

4.3 实时语音面试模块详细设计与实现

4.3.1 语音服务架构设计

实时语音面试是在文字模拟面试基础上加的增强功能, 将文本输入替换为语音交互。浏览器采集用户语音, ASR 转成文本后送入答题流程; 系统的题目和反馈经 TTS 合成为语音播放。语音能力全部放在独立部署的 Python FastAPI 微服务里, Spring Boot 后端仅负责代理转发, React 前端负责状态管理和音频播放, 三者协作关系如图 8 所示。

前端用浏览器 MediaRecorder API 采集语音, 把音频 Blob 通过 HTTP POST 发到后端 InterviewController 的 ASR 接口。后端不直接跑语音模型, 而是代理转发给 Python 语音服务, 拿到识别文本后再返回前端。TTS 流程则相反: 前端将待播报文本发送至后端 TTS 接口, 后端从语音服务拿到音频字节流, 以 audio/wav 直接返回给前端播放。

独立部署 Python 微服务, 而非在 Spring Boot 内部集成, 原因有三: ASR 和 TTS 模型深度依赖 Python 生态 (PyTorch、ONNX Runtime、soundfile 等), 和 Java 集成成本高且稳定性差; 语音模型体积大, 独立部署方便资源隔离和弹性伸缩; 语音模块是可选增强组件, 即使该服务不可用, 亦不影响文字模拟面试的正常运行, 用户可随时切换回纯文字模式。

语音技术栈的选择直接影响面试体验。Allbert 等人基于超过 30 万次 AI 面试的实证研究表明, 不同 STT×LLM×TTS 组合在对话质量与技术准确性上差异明显, 但客观质量指标和用户满意度之间只呈弱相关 ($r \approx 0.07-0.08$), 说明体验优化不能只盯着单一技术模

块的性能数字。

4.3.2 ASR 语音识别实现

语音服务基于 FastAPI 构建，主入口 voice-service/main.py。服务启动时按环境配置加载 sherpa-onnx 的离线语音识别器。sherpa-onnx 是一个本地运行的语音识别工具包，用 ONNX 和 ONNX Runtime 实现，识别过程不需要联网，全部在本地完成。本项目采用 Paraformer 中文非流式 ASR 模型，由阿里达摩院开源，通过 sherpa-onnx 进行 ONNX 格式封装。模型加载参数：sample_rate=16000、feature_dim=80、decoding_method="greedy_search"、provider="cpu"、num_threads=4，这些参数与模型训练时的配置严格对应。研究表明，"结合 Conformer 编码器与 N-gram 语言模型的方案在中文语音识别任务上具有较好的准确率与实时性平衡"^[18]。有综述对注意力机制在 ASR 系统中的应用进行了系统梳理，"梳理了注意力模型在离线与流式语音识别场景中的发展脉络"^[26]。此外，有研究提出轻量化双向 GRU 与 DCNN 混合架构，"在 CHiME-5、TED-LIUM 和 LibriSpeech 数据集上分别取得了 34.65%、10.65%和 10.08%的词错误率"^[27]，本系统选用的 Paraformer 模型属于同一技术路线。

/asr 接口接收前端传来的音频文件，读取数据后创建识别流，执行识别并返回文本，核心代码如下：

```
stream = asr_recognizer.create_stream()
stream.accept_waveform(sample_rate, samples)
asr_recognizer.decode_stream(stream)
return {"text": stream.result.text.strip()}
```

sherpa_onnx.OfflineRecognizer 采用离线批处理模式，一次性读完整段音频再识别。该模式适用于面试答题的交互方式：用户说完一段话点"停止录音"，前端统一提交完整音频并等待识别结果。比流式识别实现简单，不用处理复杂的流式状态，同等模型下准确率通常也更高。read_audio()内部用 soundfile 读 WAV 文件，转成 float32 数组供模型使用。

Spring Boot 后端通过 InterviewController 暴露 ASR 代理接口，统一处理鉴权和转发。VoiceServiceClient 基于 RestTemplate 实现，把音频以 multipart/form-data 格式转发到 Python 服务的/asr 接口。代理模式的好处是前端不需要知道语音服务的内网地址，所有通信均经由后端统一网关，日志审计、限流保护与错误降级均在此层实现。

4.3.3 TTS 语音合成实现

TTS 采用 MeloTTS，支持多语言，中文合成质量良好，在 CPU 上运行速度可满足需

求。高洁等人提出的多尺度表现力汉语语音合成方法通过同时建模全局韵律和音素级基频信息，显著提升了合成语音的自然度^[19]，MeloTTS 在中文语音合成上的声学建模策略和这个思路一脉相承。服务启动时加载中文模型和 speaker 配置，默认说话人 ID 为 ZH，语速 speed=1.0，推理设备 CPU。

面试题目和评估反馈的原始文本可能带着 Markdown 格式、特殊符号、换行符或者英文技术术语，直接塞给 TTS 模型容易出现异常停顿、吞字、发音不自然这些问题。语音服务实现了 clean_for_tts() 文本清洗函数，核心规则包括：去除 Markdown 引用符>、去掉分隔线、换行换成空格、顿号和英文逗号统一换成中文逗号、连续英文数字转小写、合并连续空白。清洗完的文本断句更自然，停顿位置更符合中文听觉习惯。

/tts 接口接收文本和可选的 speakerId，调 tts_model.tts_to_file() 合成 WAV 文件，读取内容以二进制流返回。MeloTTS 目前只支持输出到文件路径，接口内部用 tempfile.NamedTemporaryFile 建临时 WAV 文件，finally 块确保临时文件被删掉，不占磁盘。

后端 TTS 代理接口把音频流直接透传给前端，响应头设 Content-Type: audio/wav 和 Content-Disposition: inline。前端收到音频字节后，用 URL.createObjectURL(new Blob([audioBytes], { type: 'audio/wav' })) 创建临时 URL 赋给 Audio 对象播放，播完后 URL.revokeObjectURL 释放，防内存泄漏。

4.3.4 前端实时语音交互设计

在 Web 实时通信技术演进层面，传统 Web 应用主要基于轮询机制，即浏览器按特定时间间隔向服务器反复发起 HTTP 请求以获取数据，然而这种请求的 Header 通常较为冗长，造成了不必要的带宽开销，且难以满足低延迟实时交互的需求。HTML5 规范提出的 WebSocket 协议旨在实现客户端与服务器之间的全双工通信，通过单个 TCP/IP 连接提供完整的异步双工流通信，使得 Web 应用无需每次都发起 HTTP 请求来建立连接，仅需一次合法握手即可建立持久连接，从而提供精确、高效的流事件传输，将传输延时降至可忽略的程度。这种从轮询到全双工实时通信的技术演进，为实时语音交互等低延迟场景奠定了协议基础。

InterviewRealtimePanel.tsx 是实时语音面试的核心 UI 组件，定义了完整的语音交互状态机，phase 字段管着 7 个互斥状态：图 14 展示了实时语音面试的实际交互界面，呈现语音播报、录音状态和波形可视化等核心交互元素。tts：系统正在语音播报题目。播放结束后 audio.onended 回调自动进入 prep。prep：准备倒计时，默认 3 秒，给用户留思考时间。倒计时结束自动进入 recording。recording：录音中，用户对着麦

克风回答。点"停止录音"或检测到静音超时后进入 transcribing。transcribing：音频已上传，等 ASR 识别结果返回。submitting：识别文本已作为答案提交到面试后端，等系统生成回复或加载下一题。completed：这轮交互完成，展示系统反馈，自动判断是否要触发下一轮 TTS 或结束面试。

录音功能基于自定义 Hook useAudioRecorder，底层调用 navigator.mediaDevices.getUserMedia({ audio: true }) 获取麦克风权限，用 MediaRecorder 以 audio/webm 格式录音。录音过程中用 Web Audio API 的 AnalyserNode 实时获取音频时域数据，驱动 Canvas 绘制波形，大约 30 帧每秒，给用户直观的录音反馈。停止录音时调用 mediaRecorder.stream.getTracks().forEach(track => track.stop()) 立刻释放麦克风，不让指示灯一直亮着。TTS 播放由自定义 Hook useTtsPlayer 管理。系统需要播报题目或反馈时，组件调用 TTS 接口拿音频 Blob，创建 Audio 对象播放，播完自动触发状态流转。值得注意的是，TTS 播放完成后自动进入准备倒计时，无需用户手动点击“下一题”，由此保证语音交互的连贯性。。TTS 服务异常或请求失败时，系统捕获错误自动降级为纯文字展示，面试不会卡住。一轮完整的实时语音面试流程如下：（1）系统调用 TTS 合成并播放当前题目（tts → prep）。（2）准备倒计时 3 秒结束，页面显示"请开始回答"并自动开始录音（prep → recording）。（3）用户答完点停止，进入上传和识别阶段（recording → transcribing）。（4）前端把音频 Blob 发给后端 ASR 接口，拿到识别文本（transcribing）。（5）识别文本作为答案提交到答题接口，后端返回系统回复（submitting → completed）。（6）还有下一题就 TTS 播报，进下一轮（completed → tts）；没有就跳报告页。

4.4 知识库管理模块详细设计与实现

4.4.1 模块架构设计

知识库管理模块面向希望基于私有资料进行备考的用户，支持将整理好的文档上传至系统，并针对文档内容进行定向问答。模块覆盖了从文档上传、文本分块、向量存储到检索召回和答案生成的完整 RAG 链路^[12]。大语言模型 (Large Language Models, LLMs) 凭借出色的生成能力已成为自然语言处理领域的核心技术，然而其固有的幻觉 (Hallucination) 倾向、知识时效性不足以及推理过程不透明且难以追溯等问题，限制了其在高可靠性场景中的实际应用。检索增强生成 (Retrieval Augmented Generation, RAG) 技术作为应对上述挑战的有效方案，通过整合外部数据库的实时信息，显著提升了生成结果的准确性与可信度，尤其在知识密集型任务中表现优异；该技术巧妙融合了大语言模型的内置参数知识与外部动态信息资源，实现了知识的持续更新与领域特定信

息的无缝衔接。基于此，本模块采用 RAG 架构以兼顾生成质量与知识时效性。这个方向已有一些可参考的工程实践：Wang 等人提出的 HyperSynergyX 框架通过双偏置随机游走超图（DBRWH）建模高阶关系，结合知识图谱增强 RAG 实现了协同推理和可解释性增强^[28]；Kyurkchiev 等人用 Qdrant 向量数据库做语义搜索实验，在和全文检索、关键词检索、BM25 的对比中，语义搜索精确率超过了 90%^[29]。各环节依赖关系如图 9 所示。

数据存储采用“双轨制”：文档元数据（名称、分类、上传时间、向量化状态、分块数量等）存 PostgreSQL 的 knowledge_bases 表；文档内容分块向量化后存 vector_store 表，Spring AI 首次启动时自动创建，embedding 字段用 pgvector 的 vector 类型，支持余弦相似度检索。这种设计让关系型查询和相似度检索各自在最合适的存储引擎上跑：PostgreSQL 负责元数据的 CRUD 和过滤，pgvector 负责高维向量的近似最近邻搜索。pgvector 支持 HNSW 和 IVFFlat 两种索引策略，在享受 PostgreSQL 事务安全性的同时实现高效向量检索。

4.4.2 文档上传与异步向量化

查重 100%
随着互联网技术的飞速发展和社交媒体的爆炸式增长，数据量与信息量暴增。随着各种信息铺天盖地的到来，数据飞速更新。对于用户来说，想要准确地获得对于自己有价值的信息越来越难。因此自动化的文本分类与挖掘的技术应运而生，也随着时代的发展变得越来越重要。**查重 58%**
利用 TF-IDF 词向量化模型来提取文本的特征向量，通过文本分类技术快速自动化地处理海量数据，比人工处理更加的高效智能。上述文本向量化的思想在现代知识库系统中得到延续与发展：**查重 41%**通过将非结构化文档转化为高维向量表示，系统能够在海量信息中实现高效的自动化检索与问答。本知识库模块的文档向量化流程正是基于这一技术路径展开。

用户在 KnowledgeBaseUploadPage 选好文件填完分类后，调 POST /api/knowledge-bases/upload。上传流程和简历上传类似：文件校验、S3 存储、元数据落库、异步向量化任务投递。KnowledgeBaseUploadService 把文件存 S3 后在 knowledge_bases 表插入一条记录，vectorStatus 初始为 PENDING，chunkCount 初始为 0，然后往 Redis Stream 发向量化任务，消息体包含知识库 ID 和 S3 存储路径。

向量化任务消费者 VectorizeStreamConsumer 从 Redis Stream 拿到消息后按以下步骤执行：把文档状态改成 PROCESSING；从 S3 下载文件流，调 Apache Tika 解析纯文本；用 Spring AI 的 DocumentSplitter 做滑动窗口分块，默认每块约 1000 个 Token，相邻块之间约 200 个 Token 重叠，保证语义连续性；调 vectorStore.add(chunks)，Spring AI 自动调 Embedding API 把每个分块转成向量写入 pgvector 的 vector_store 表。完成后 vectorStatus

改成 COMPLETED，实际分块数写入 chunkCount。

异步处理在这里是必须的：几十页的技术文档，Embedding 调用可能要几十秒甚至几分钟，同步等的话上传接口直接卡死，体验极差。Redis Stream 异步处理让上传接口毫秒级返回，前端轮询看向量化进度就行。

4.4.3 RAG 检索增强问答实现

KnowledgeBaseController 提供了两个问答接口：同步接口 /api/knowledge-bases/query 返回完整答案字符串；SSE 流式接口 /api/knowledge-bases/query/stream 逐字推送，实现打字机效果，等待体验好不少。KnowledgeBaseQueryService 是 RAG 问答的核心，负责检索上下文构建、向量检索、Prompt 组装和答案生成。非流式问答的核心流程：根据用户问题构建检索请求，设 TopK=5、相似度阈值 0.7；若用户指定了知识库范围就加元数据过滤器，只在选定的知识库内检索；调向量存储的相似度搜索从 pgvector 召回相关文档片段；最后把召回片段拼成上下文，和用户问题一起组装成 Prompt 发给 LLM。TopK=5 能覆盖足够信息量，又不会因上下文太长稀释重点；相似度阈值 0.7 过滤掉低质量结果，防止无关片段干扰模型判断；元数据过滤还支持多知识库联合问答这种灵活场景。RAG 的 System Prompt 是控制回答质量、压制模型幻觉的关键[14]。系统 Prompt 明确了三条约束：严格根据提供的文档片段作答；文档里没有相关信息时必须明确告知用户“根据现有知识库内容无法回答该问题”，不得编造；回答简洁准确，可以适当引用原文。这三条约束加上相似度阈值过滤，把 RAG 回答的可信度和可追溯性提升了不少。

在服务器推送技术层面，传统 Web 应用依赖轮询方式获取数据，浏览器需按固定间隔反复发起 HTTP 请求，冗长的请求头带来了显著的带宽开销与连接负担。HTML5 规范中定义的 WebSocket 采用单 Socket 全双工传输机制来“推送”和“拉取”信息，不仅避免了 Comet 中存在的连接与可移植问题，还能够提供比 Ajax 轮询更高效的解决方案，为浏览器提供精确、高效的流事件传输，保证传输延时微乎其微。与 WebSocket 同属 HTML5 实时通信体系的 SSE (Server-Sent Events) 技术同样基于持久连接实现服务器单向推送，适用于文本流式输出场景。流式问答接口直接返回 Flux<String>。

4.4.4 前端知识库页面设计

KnowledgeBaseManagePage 用卡片表格展示所有已上传的知识库文档，每行显示文档名称、分类、上传时间、向量化状态标签和分块数量。

图 15 展示了知识库管理页面，用户可在该页面查看已上传文档的元信息、向量化状态，并执行删除或重新向量化操作。向量化失败的文档状态标签显示红色 FAILED，用户可以点“重新向量化”再次触发异步任务。顶部有搜索框和分类筛选器，文档多的时

候方便定位。

KnowledgeBaseQueryPage 用类似 ChatGPT 的对话布局：左边是会话列表和知识库选择区，用户勾选一个或多个已向量化的知识库作为当前问答来源；右边是消息区，用户消息在右侧气泡，AI 回复在左侧。

图 16 展示了 RAG 知识库问答的实际对话界面，用户可选择知识库来源并发起基于文档内容的智能问答。AI 回复过程中文字末尾有闪烁光标表示流式接收中，流结束光标消失，用户可以继续输入。

原生 EventSource 不支持 POST，前端用 fetch 发请求，通过 `response.body.getReader()` 拿 `ReadableStream` 读取器，配合 `TextDecoder` 逐块解码 SSE 数据。每次读到新数据后按 `\n\n` 分割成独立 SSE 事件，解析出 `data:` 内容追加到当前 AI 消息状态，触发 React 重新渲染。这套做法虽然比原生 EventSource 麻烦一点，但完美支持 POST 和自定义请求头，是现代 Web 应用消费 SSE 流的标准方式。

RAG 问答支持多会话管理，用户可以建多个独立会话。系统在 `rag_chat_sessions` 表插记录，会话标题初始化为问题的前 20 个字。每次问答后，用户问题和 AI 回答存进 `rag_chat_messages` 表，更新会话的 `messageCount` 和 `updatedAt`。会话支持重命名、置顶和删除，列表按 `isPinned` 降序和 `updatedAt` 降序排，置顶的会话始终在最上面。

4.5 本章小结

本章围绕简历管理、模拟面试、实时语音面试和知识库管理四个模块，详细说明了各自的设计决策与关键实现。简历管理模块通过内容哈希去重和 Redis Stream 异步分析，在保证低延迟的同时实现了对多格式简历的自动解析与评分。模拟面试模块基于简历内容生成个性化题目，采用线性追问流提升真实感，并以分批评估加二次汇总的策略有效解决了长文本场景下的 Token 溢出问题。实时语音面试模块以独立 Python 微服务的形式集成了 `sherpa-onnx` ASR 和 `MeloTTS` TTS，配合前端完整的状态机实现了连贯的语音交互闭环。知识库管理模块则基于 `pgvector` 完成了文档分块、向量存储与相似度检索，结合 SSE 流式输出为用户提供了接近实时问答工具的体验。以上四个模块的技术实现共同支撑了 InterviewGuide 平台的核心能力。

5 系统测试与分析

软件测试是软件质量的重要保证。从软件工程学的视角来看，软件测试就是为了发现错误而执行程序的过程。在软件开发实践中，应当把“尽早地和不断地进行软件测试”作为软件开发者的座右铭，不应将软件测试仅仅视为软件开发的一个独立阶段，而应将其贯穿到软件开发的各个阶段中。在测试方法层面，黑盒测试（或称功能测试）不

考虑程序的内部结构和表现，其目的在于确定程序的输入与输出是否与其规格一致。基于上述测试原则，本章将围绕系统测试展开，通过科学的测试方法保障软件质量、验证需求满足度。

5.1 测试环境

5.1.1 硬件环境

查重 60%

查重 49%

系统测试是保障软件质量、验证需求满足度的关键环节。本章介绍测试环境的软硬件配置，设计并执行功能测试与性能测试用例，最后对测试结果进行分析总结。

测试与部署均在本地机器上进行，支持两种硬件环境：

Mac Mini M4

设备/资源	配置说明
机器型号	Apple Mac Mini M4
CPU	Apple M4 芯片（10 核 CPU）
内存	24 GB 统一内存
存储	256 GB NVMe SSD

Windows 工作站

设备/资源	配置说明
机器型号	AMD Ryzen 5 6800H 台式机/笔记本
CPU	AMD Ryzen 5 6800H（8 核 16 线程）
内存	16 GB DDR5
存储	512 GB NVMe SSD

两种环境下均通过 Docker Compose 一键启动全部服务，语音服务与主服务共用同一台机器，不需要额外配置内网通信。

5.1.2 软件环境

软件/组件	版本说明
-------	------

软件/组件	版本说明
操作系统	macOS 15.x / Windows 11
JDK	OpenJDK 21
Python	3.11
Node.js	20.x
Docker Desktop	4.x (包含 Docker Compose)
PostgreSQL	16.x (Docker 容器) , 含 pgvector 扩展
Redis	7.x (Docker 容器)
MinIO (对象存储)	latest (Docker 容器)

5.1.3 测试工具

接口测试用 Postman 和 Apifox , 压力测试用 Apache JMeter 5.6.3 , 前端调试用 Chrome DevTools 和 React Developer Tools , 数据库监控用 pgAdmin 4 和 RedisInsight , 日志分析用 ELK Stack (Elasticsearch + Logstash + Kibana) 。

5.2 功能测试

功能测试采用黑盒测试方法，依据第 2 章的需求分析，对四大核心模块设计测试用例，每个功能点按"测试项—测试步骤—预期结果—实际结果—测试结论"的格式记录。

5.2.1 简历管理模块功能测试

测试用例 1：简历上传与格式校验

项目	内容
测试目的	验证系统能否正确接收并校验简历文件
测试步骤	1. 进入上传页面; 2. 分别上传 PDF、DOCX、TXT 格式的简历; 3. 上传一个 15MB 的 PDF 文件; 4. 上传一个 JPG 图片文件
预期结果	PDF、DOCX、TXT 上传成功; 15MB 文件提示"文件过大"; JPG 文件提示"格式不支持"

实际结果	所有格式校验均符合预期
测试结论	通过
测试用例 2：重复上传检测	
项目	内容
测试目的	验证基于内容哈希的去重机制是否生效
测试步骤	1. 上传简历 A（PDF）；2. 将简历 A 重命名为 A2 后再次上传； 3. 观察返回结果和数据库记录数
预期结果	第二次上传返回已有记录，resumes 表中仅存在 1 条记录， accessCount 增加为 2
实际结果	返回状态为 COMPLETED 的已有记录，数据库无新增记录
测试结论	通过
测试用例 3：异步 AI 分析状态流转	
项目	内容
测试目的	验证简历分析的状态机是否正确流转
测试步骤	1. 上传一份新简历；2. 观察详情页状态变化；3. 查看数据库 analyze_status 字段
预期结果	上传成功后状态为 PENDING，2 秒内变为 PROCESSING，10-30 秒 后变为 COMPLETED，页面展示 5 维度评分雷达图
实际结果	状态正常流转，平均分析耗时约 18 秒
测试结论	通过
测试用例 4：PDF 报告导出	
项目	内容

测试目的	验证简历分析报告能否正确导出为 PDF
测试步骤	1. 打开一份分析完成的简历详情；2. 点击"导出 PDF 报告"按钮； 3. 打开下载的 PDF 文件检查内容完整性
预期结果	PDF 包含简历基本信息、雷达图、各维度得分、优点列表和改进建议
实际结果	PDF 内容完整，图表清晰，排版正确
测试结论	通过

5.2.2 模拟面试模块功能测试

测试用例 5：面试会话创建与题目生成

项目	内容
测试目的	验证系统能否基于简历内容生成个性化面试题
测试步骤	1. 选择一份 Java 后端开发简历；2. 创建一场包含 5 道题的面试； 3. 查看返回的题目列表
预期结果	5 道题目均与简历内容相关，至少包含 2 道项目经验题和 1 道技术基础题
实际结果	生成题目与简历高度匹配，包含 Spring Boot、Redis、MySQL 等技术点
测试结论	通过

测试用例 6：答题提交与状态更新

项目	内容
测试目的	验证面试答题流程和会话状态更新
测试步骤	1. 创建面试会话；2. 逐题作答并提交；3. 观察进度条和当前题号变化；

项目	内容
步骤	4. 提交最后一题
预期结果	每提交一题进度条递增，题号+1，最后一题提交后状态变为 COMPLETED 并触发评估
实际结果	答题流程顺畅，状态更新及时
测试结论	通过

测试用例 7：智能追问展示

项目	内容
测试目的	验证智能追问是否正常展示和作答
测试步骤	1. 开启"启用追问"配置创建面试；2. 回答第一题主问题；3. 查看是否出现追问；4. 回答追问并提交
预期结果	主问题提交后出现追问内容，追问答案与主问题答案合并存储
实际结果	追问正常出现，答案合并正确
测试结论	通过

测试用例 8：面试评估与报告

项目	内容
测试目的	验证面试完成后的自动评估和报告生成
测试步骤	1. 完成一场包含 8 道题的模拟面试；2. 等待评估完成；3. 查看面试报告页和 PDF 导出
预期结果	每道题均有得分和反馈，报告页展示总体评价、优势分析和改进建议

实际结果	8 道题评估结果完整，总分计算正确，PDF 导出正常
测试结论	通过

5.2.3 实时语音面试模块功能测试	
测试用例 9：语音识别（ASR）功能	

项目	内容
测试目的	查重 69% 验证语音输入能否正确转换为文本
测试步骤	1. 进入实时语音面试模式；2. 用普通话朗读一段技术介绍（约 100 字）；3. 停止录音，观察识别结果
预期结果	识别文本与口述内容高度一致，无明显错别字
实际结果	识别结果与口述内容高度一致，个别专业术语（如"Redis"）偶有识别偏差
测试结论	通过

测试用例 10：语音合成（TTS）功能	
---------------------	--

项目	内容
测试目的	验证系统回复能否正确合成为语音
测试步骤	1. 进入实时语音面试模式；2. 听完系统播报的题目；3. 语音回答后听取反馈播报
预期结果	语音播报自然流畅，语速适中，无明显机械感或断句错误
实际结果	TTS 播报效果良好，中文语音自然流畅，无明显机械感
测试结论	通过

测试用例 11：语音状态管理

项目	内容
测试目的	验证语音交互各状态的流转是否正确
测试步骤	观察一轮完整的语音面试流程 (tts → prep → recording → transcribing → submitting → completed)
预期结果	各阶段 UI 提示清晰，状态切换无卡顿，倒计时准确
实际结果	状态流转顺畅，TTS 播放结束后能正确触发准备倒计时
测试结论	通过

5.2.4 知识库管理模块功能测试

测试用例 12：文档上传与向量化

项目	内容
测试目的	验证知识库文档的上传和异步向量化流程
测试步骤	1. 上传一份 20 页的 PDF 技术文档；2. 查看列表页向量化状态；3. 等待状态变为 COMPLETED
预期结果	上传成功后状态为 PENDING，随后变为 PROCESSING，1-3 分钟后变为 COMPLETED，chunkCount 大于 0
实际结果	20 页 PDF 向量化耗时约 90 秒，分块数量为 47
测试结论	通过

测试用例 13：RAG 问答准确性

项目	内容
测试目的	查重 45% 验证基于知识库的问答能否准确召回文档内容
测试步骤	1. 上传一份关于 Spring Boot 事务管理的文档; 2. 提问"Spring Boot 中@Transactional 注解的传播行为有哪些? "; 3. 观察回答内容
预期结果	回答准确包含 REQUIRED、REQUIRES_NEW、NESTED 等传播行为, 并正确引用文档中的定义
实际结果	回答准确, 引用了文档中的原文说明
测试结论	通过

测试用例 14: SSE 流式响应

项目	内容
测试目的	验证知识库问答的流式返回效果
测试步骤	1. 在问答页输入一个问题; 2. 观察 AI 回复的展示方式; 3. 用 Chrome DevTools 的 Network 面板检查响应类型
预期结果	文字逐字出现, Network 面板显示 Content-Type 为 text/event-stream, 响应以 data:开头
实际结果	打字机效果正常, SSE 响应格式正确
测试结论	通过

测试用例 15: 超出知识边界的问题处理

项目	内容
测试目的	验证系统对知识库外问题的兜底策略
测试步骤	1. 仅上传 Java 后端文档; 2. 提问"请介绍一下 Python 的 GIL 机制"
预期结果	系统回答"根据现有知识库内容无法回答该问题"或类似提示
实际结果	系统正确拒绝了超出知识边界的问题
测试结论	通过

5.3 性能测试

5.3.1 接口响应时间测试

查重 43%

用 Postman 对关键接口做响应时间测试，每个接口连续调用 10 次取平均值，结果如表 8 所示。

表 8 关键接口响应时间测试结果

接口	平均响应时间	最大响应时间	最小响应时间	是否符合需求
简历上传	245 ms	312 ms	198 ms	是
创建面试会话	3,850 ms	4,120 ms	3,600 ms	是 (异步生成题目)
获取当前题目	28 ms	45 ms	18 ms	是
提交答案	56 ms	89 ms	38 ms	是
获取面试报告	42 ms	67 ms	31 ms	是
简历分析状态查询	22 ms	35 ms	15 ms	是
知识库问答 (同步)	4,200 ms	5,100 ms	3,800 ms	是 (LLM 生成)

接口	平均响应时间	最大响应时间	最小响应时间	是否符合需求
知识库问答 (SSE 首字)	850 ms	1,200 ms	650 ms	是

查重 42%

由表 8 可知, 不涉及大语言模型实时调用的接口平均响应时间均在 500ms 以内, 满足第 2 章的性能需求。创建会话和同步问答由于要等 LLM 推理, 响应时间在 3-5 秒之间, 属于模型侧耗时决定的范围。上传和提交最后一题这类异步任务接口通过立即返回状态, 把实际耗时操作转到后台, 前端无需持续等待。

5.3.2 并发压力测试

用 Apache JMeter 对上传接口和答题接口做并发压力测试, 模拟 50 个虚拟用户同时操作。

(1) 简历上传接口并发测试

测试场景: 50 个虚拟用户在 10 秒内渐进加压, 持续 60 秒, 每人上传一份不同的简历 PDF。

测试结果:

总请求数: 1,247

平均响应时间: 1,850 ms

错误率: 0.08% (1 个请求因网络超时失败)

吞吐量: 19.8 req/s

99%分位响应时间: 4,200 ms

并发上传时, 文件解析和 S3 上传本身有 I/O 开销, 平均响应时间比单用户场景高一些, 但错误率极低, 没有出现资源耗尽或崩溃。陈建海等人的研究表明, 基于微服务架构的 B/S 系统通过合理的服务拆分和负载均衡能够有效提升并发处理能力^[20], 本次压测结果和这一结论一致。测试中 @RateLimit 注解被绕过 (用了不同客户端标识), 主要压力来自文件处理线程池。当前配置下系统可稳定支撑约 20 req/s 的简历上传峰值。

(2) 答题提交接口并发测试

测试场景: 基于已创建的 100 个面试会话, 50 个虚拟用户循环提交答案, 持续 60 秒。

测试结果:

总请求数: 3,856

平均响应时间: 68 ms

错误率: 0%

吞吐量: 62.2 req/s

99%分位响应时间: 145 ms

答题提交接口主要操作 Redis 缓存和简单数据库写入, 性能很好。62 req/s 的吞吐量远超普通用户场景的需求, 该接口在高并发下没什么压力。

5.3.3 长文本与分批评估测试

为验证分批评估策略在长文本场景下的稳定性, 设计了专门的测试用例。

测试场景: 创建一场包含 15 道题的面试, 每道题均用超过 800 字的长文本作答, 观察评估过程能否正常完成。

测试结果:

评估总耗时: 约 42 秒

LLM 调用次数: 3 次分批评估 + 1 次二次汇总 = 4 次

Token 溢出错误: 0 次

最终报告生成: 正常

对比实验: 关闭分批评估, 尝试把 15 道题和答案一次性传给 LLM 评估。

对比结果:

DashScope qwen-plus 报错: "输入长度超出最大上下文长度"

评估失败, 状态变为 FAILED

分批评估策略对于长文本、多题目的面试场景是必要的^[2]——若未启用则直接失败, 启用后评估成功率由 0%提升至 100%, 结论明确。

5.3.4 向量检索性能测试

测试数据: 一份被切成 200 个 chunk 的技术文档。

测试场景: 连续进行 100 次向量相似度查询, 测量平均检索耗时。

测试结果:

平均检索耗时: 35 ms

最大检索耗时: 78 ms

最小检索耗时: 18 ms

召回准确率 (Top-5) : 92%

pgvector 支持 IVF 和 HNSW 两种近似最近邻索引^[13], 在本系统的数据规模下平均检索耗时只有 35ms, 完全能支撑实时问答的需求。92%的 Top-5 召回准确率说明向量检索

能有效定位与问题相关的文档片段。

5.4 测试结果分析

5.4.1 功能测试总结

本次功能测试覆盖四大核心模块，15 个测试用例全部通过。

简历管理方面：文件上传、格式校验、去重、异步分析和 PDF 导出运行稳定。SHA-256 内容哈希去重能拦截重复文件，改了文件名也不会误判；Redis Stream 异步架构让上传接口保持在毫秒级，前端不会因为 AI 分析耗时卡住。

模拟面试方面：题目生成和简历内容的匹配度比较高，测试中的 Java 后端简历能稳定输出和 Spring Boot、Redis、MySQL 相关的题目。智能追问的线性流设计降低了状态管理复杂度，答题提交的响应和状态更新也比较及时。

实时语音面试方面：sherpa-onnx 对普通话的识别准确率令人满意，约 100 字的技术介绍朗读后识别结果和原文高度一致，虽然"Redis"这类英文术语偶有偏差，但不影响整体理解。许鸿奎等人的研究表明，结合 Conformer 与 N-gram 的语音识别方案在中文场景下准确率有保障^[18]，测试结果和该结论一致。MeloTTS 合成的中文语音自然度不错，多尺度表现力建模在提升自然度方面发挥了作用^[19]。前端状态机各阶段切换顺畅，TTS 播放结束后能正确触发倒计时。

知识库管理方面：20 页 PDF 异步向量化耗时约 90 秒，分块数量符合预期。RAG 问答对事务管理相关问题的回答准确引用了文档原文；超出知识边界的问题，System Prompt 约束生效，模型明确告知无法回答而不是乱编。SSE 流式输出的打字机效果正常，首字延迟约 850ms。

5.4.2 性能测试总结

整体性能表现符合预期。

接口响应方面：不涉及大模型调用的接口普遍在 500ms 以内。创建面试会话和同步知识库问答要等 LLM 推理，响应时间在 3-5 秒之间，是模型侧决定的，属于可接受范围。SSE 流式问答首字延迟约 850ms，用户能较快看到文字开始输出，比等整段答案返回的体验好不少。

并发能力方面：50 并发下答题提交接口达到 62 req/s，高并发下表现稳定。简历上传接口因为有文件解析和对象存储 I/O，吞吐量约 20 req/s，这是文件处理类接口的正常水平。后续如果要提升峰值承载，可以考虑分片上传或增加存储带宽。

长文本稳定性方面：分批评估策略效果很明显：不开启时 qwen-plus 直接报上下文超限；开启后 3 次分批加 1 次汇总顺利完成，成功率从 0%到 100%。这个数字说明分批

评估是大规模面试场景下系统稳定运行的必要条件，不是可选优化。

向量检索方面：pgvector 的 Top-5 相似度查询平均 35ms，召回准确率 92%，能在用户发问后迅速定位相关文档片段，为 RAG 问答的实时性打好了基础。

5.4.3 存在的问题与改进方向

测试过程也暴露了几个可以继续优化的地方。

ASR 对英文技术术语的识别准确率有下降，“Kubernetes”、“Elasticsearch”这类词在普通话环境下偶尔会被识别成发音相近的中文词。后续可以在 ASR 结果后加一层术语纠错模块，或者换一个支持中英混说的 ASR 模型。

TTS 的首包延迟大约 1-2 秒，根源在于 MeloTTS 目前只能先把音频写到临时文件再读取返回。如果后续能找到支持流式输出的 TTS 方案，把音频流直接回传给前端，等待时间可以明显缩短。

查重 52%
高并发上传时，文件解析可能成为瓶颈。目前 Tika 解析在上传接口里同步执行，后续可以把这步也迁到异步消费者里，或者用对象存储的 S3 事件通知机制触发后台处理。

RAG 问答支持多文档关联，但文档之间的交叉引用和关系推理能力还比较有限。后续可以探索 GraphRAG，从文档中自动抽取实体和关系建知识图谱，结合图结构和向量混合检索，提升复杂问题的回答准确性和可解释性。

5.5 本章小结

本章首先介绍了测试环境的软硬件配置，并说明了所采用的测试工具；随后设计并执行了覆盖四大模块的 15 个功能测试用例，全部通过。在性能测试方面，分别针对关键接口响应时间、50 并发压力、长文本面试评估和向量检索准确率四个维度进行了测试与分析。实验数据表明：非 LLM 接口响应时间均控制在 500ms 以内，答题提交接口吞吐量达 62 req/s，分批评估策略将 15 道长文本面试的评估成功率从 0%提升至 100%，pgvector 向量检索平均耗时 35ms 且 Top-5 召回准确率达 92%。查重 46% 综合来看，系统在功能正确性、并发稳定性和长文本处理能力上均达到了预期设计目标。

6 总结与展望

6.1 工作总结

本课题设计并实现了一个面向求职者和 HR 的智能面试辅助平台——InterviewGuide，旨在利用大语言模型、RAG 和语音技术解决传统招聘流程中简历筛选效率低下、人岗匹配度不足以及求职者缺乏个性化面试练习工具等问题。

在系统架构方面，本系统采用 B/S 三层架构，前端基于 React 18.3 与 Vite 开发，样式方案采用 Tailwind CSS；后端主服务基于 Spring Boot 和 Java 21 构建，语音服务以 Python

FastAPI 独立部署为微服务；数据层选用 PostgreSQL 配合 pgvector 扩展同时承担关系型数据与向量数据的存储，Redis Stream 作为异步消息队列支撑简历分析、面试评估和知识库向量化等耗时任务的异步调度。这种微服务解耦设计确保了语音等增强功能不可用时不影响核心业务的正常运行。

查重 54%

在功能实现方面，本系统包含以下四个核心模块：

(1) 简历分析模块：支持 PDF、DOCX、DOC、TXT 等多种格式的简历上传与解析，采用 SHA-256 内容哈希机制实现文件去重，通过大语言模型从内容完整性、结构清晰度、技能匹配度、表达专业性和项目经验五个维度生成结构化评估报告，并支持导出 PDF 格式分析报告。

(2) 模拟面试模块：系统基于候选人简历内容动态生成个性化面试题目，支持多轮智能追问，构建线性问答流；采用分批评估策略解决长文本场景下的 Token 溢出问题，并通过二次汇总生成总体评价与改进建议。

(3) 实时语音面试模块：集成 sherpa-onnx 实现本地离线中文语音识别，集成 MeloTTS 实现中文语音合成，前端实现完整的语音交互状态机，覆盖 TTS 播报、准备倒计时、录音、识别、提交和完成等阶段，实现了连贯的语音交互闭环。

(4) 知识库问答模块：支持用户上传专业文档构建个人知识库，系统对文档进行异步分块与向量化存储；问答时通过向量相似度检索召回相关片段，结合大语言模型生成回答，并以 SSE 流式方式返回，实现打字机效果。

在系统测试方面，15 个功能测试用例全部通过。性能测试数据表明：答题提交接口在 50 并发条件下吞吐量达 62 req/s，pgvector 向量检索平均耗时 35ms，Top-5 召回准确率达 92%，系统整体性能满足实际应用需求。

查重 54%

6.2 项目亮点

与市面上现有方案相比，本系统在以下三个方面具有特色：

做到了从解析到面试的“端到端”闭环。现有招聘工具或学术方案（如 Qin 等提出的 TAPIFNN）^[6] 或者何苗的人岗匹配研究^[30]，它们大多聚焦于简历筛选准确性的研究。本系统的特色在于将简历分析识别的不足与优势直接嵌入 Prompt 模板，作为面试题的上下文。由此生成的题目具有高度个性化特征，而非通用模板化题目。

设计了一套应对 Token 限制的分批评估机制。大模型面临的主要挑战在于用户回答过长导致上下文溢出。所设计的策略为：首先将所有答题内容按批次划分，由 LLM 逐批评估，最终对各批次评估结果进行汇总，生成二次润色与总体评价。测试结果表明，未采用分批策略时一次性评估 15 道题直接报错（成功率为 0%），采用分批策略后成功

率稳定达到 100%。

实现了低成本、离线可用的语音方案。 市面上的语音 API 通常按调用次数计费，且依赖网络环境。本系统将 sherpa-onnx 部署于本地，推理均由 CPU 承担，无需额外费用，且不存在隐私风险。此外，语音服务为可选组件，即使不可用亦可自动回退至文字模式，灵活性更强。

6.3 遗憾与未来计划

虽然系统目前已可运行，但受限于开发周期与软硬件环境，仍存在若干待完善之处。

交互层面：当前面试反馈仍为单向输出模式，用户无法反驳或追问。参考陆苏于提的多智能体思路，未来可将评估报告升级为对话式交互，用户可就评分与 AI 进行深度探讨。

评估维度：目前系统还是在“读”文字。面试官其实还会看语速、停顿甚至面部表情。下一步拟尝试引入语音韵律特征乃至摄像头视觉信息，构建文本-语音-视觉的三维画像，其准确性显著优于纯文本分析。

知识库深度：当前 RAG 完全依赖语义相似度检索片段，面对需跨多个文档推理的复杂问题时表现有限。李泽鸣等提出的 MRAG 或当前广泛关注的 GraphRAG 均为可行方向。首先抽取实体构建知识图谱，继而配合向量搜索，有望提升复杂问题回答的可解释性。

公平性与适配：大模型本身可能有偏见（比如对学校、性别的歧视），这在招聘场景里很严重。未来需增设审计机制并开展对抗测试。另外，现在只有 PC 版，后续拟开发微信小程序或 H5 移动端版本，毕竟求职者更多是在碎片化时间里练习的。

综上所述，本课题完成了从架构设计、系统实现、测试验证到性能调优的完整研究周期，不仅实现了本系统的设计与落地，**亦为后续多模态评估与知识图谱增强研究积累了实践经验**

参考文献

- [1] 车万翔, 龚志成, 冯岩松, 等. 大模型时代的自然语言处理: 挑战、机遇与发展[J]. 中国科学: 信息科学, 2023, 53(09): 1645-1687.
- [2] 赵凯博, 李宇楠, 彭成, 等. 大模型发展综述: 从 BERT 到 Deepseek 的技术演进[J/OL]. 微电子学与计算机, 1-21[2026-04-18]. HYPERLINK "https://link.cnki.net/urlid/61.1123.TN.20260416.1137.010" https://link.cnki.net/urlid/61.1123.TN.20260416.1137.010.

- [3] 刘梦瑶,王武军,彭继阳,等.大模型技术及其在垂直领域应用综述[J/OL].计算机测量与控制,1-10[2026-04-18]. HYPERLINK "https://link.cnki.net/urlid/11.4762.TP.20260408.1756.022" https://link.cnki.net/urlid/11.4762.TP.20260408.1756.022.
- [4] Reimers N ,Gurevych I .Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.[J].CoRR,2019,abs/1908.10084
- [5] Rosenberger J ,Wolfrum L ,Weinzierl S , et al.CareerBERT: Matching resumes to ESCO jobs in a shared embedding space for generic job recommendations[J].Expert Systems With Applications,2025,275127043-127043.DOI:10.1016/J.ESWA.2025.127043.
- [6] QIN C, ZHU H, XU T, et al. An enhanced neural network approach to person-job fit in talent recruitment[J]. ACM Transactions on Information Systems, 2020, 38(2): 1-33. DOI:10.1145/3376927.
- [7] 蒋欣奕 . 人工智能技术在招聘流程优化中的应用效果评估 [J]. 财经界,2026,(09):174-176.DOI:10.19887/j.cnki.cn11-4098/f.2026.09.044.
- [8] 李家鹏 . 网上招聘智能推荐系统研究 [D]. 阜阳师范大学,2024.DOI:10.27846/d.cnki.gfysf.2024.000217.
- [9] YANG X. Research on personalized distance education recommendation system based on deep learning[J]. Scientific Reports, 2025, 15(1): 42158-42158. DOI:10.1038/S41598-025-26020-1.
- [10] 周崇钦.人工智能在事业单位人岗匹配中的应用[J].人力资源,2026,(04):4-6.
- [11] 陆苏于,汪力宾,沈心妮,等.基于多智能体协同的大学生就业面试训练系统"GEMINI+互感评估" workflow 设计 [J]. 电脑知识与技术,2025,21(09):68-70.DOI:10.14004/j.cnki.ckt.2025.0455.
- [12] 袁乐,刘绍华,王禹,等.大语言模型检索增强生成优化技术研究综述[J].计算机学报,2026,49(02):383-422.

- [13] 李泽鸣,王树良,尚子贺,等.多模态检索增强生成驱动文档问答综述(特邀)[J].计算机工程,2026,52(04):1-21.DOI:10.19678/j.issn.1000-3428.0260043.
- [14] 包晓明.基于检索增强生成的私有化 AI 问答系统研究[J].电子设计工程,2026,34(08):120-124.DOI:10.14022/j.issn1674-6236.2026.08.025.
- [15] 孔繁恒,高永祺,张子帅,等.基于检索与生成混合模型的个性化聊天机器人系统的设计与实现 [J]. 软 件 工 程,2022,25(11):23-27.DOI:10.19644/j.cnki.issn2096-1472.2022.011.006.
- [16] 霍福华,韩慧.基于 SpringBoot 微服务架构下前后端分离的 MVVM 模型[J].电子技术与软件工程,2022,(01):73-76.DOI:10.20109/j.cnki.ets.2022.01.018..
- [17] 胡荣,羊雪玲.基于 Spring Boot 前后端分离 Web 系统的设计与实现[J].新能源与智能网联,2024,(01):88-97.
- [18] 许鸿奎,卢江坤,张子枫,等.结合 Conformer 与 N-gram 的中文语音识别[J].计算机系统应用,2022,31(07):194-202.DOI:10.15888/j.cnki.csa.008638.
- [19] 高洁,肖大军,徐遐龄,等.多尺度富有表现力的汉语语音合成[J].数据采集与处理,2023,38(06):1458-1468.DOI:10.16337/j.1004-9037.2023.06.019..
- [20] 陈建海,陈淼,浦云明.基于微服务架构 B/S 系统的性能分析[J].计算机系统应用,2020,29(02):233-237.DOI:10.15888/j.cnki.csa.007285.
- [21] 宋琦敏.简历自动获取与信息提取系统设计与实现 [D]. 华中科技大学,2020.DOI:10.27157/d.cnki.ghzku.2020.000897.
- [22] 冯立.基于深度学习的中文简历信息实体识别研究 [D]. 武汉轻工大学,2024.DOI:10.27776/d.cnki.gwhgy.2024.000493.
- [23] 杨济萍.基于自然语言处理的简历信息抽取与识别研究 [D]. 兰州交通大

学,2022.DOI:10.27205/d.cnki.gltec.2022.000624.

- [24] 张书祥.基于 CNN 和多头 Attention 融合算法的简历信息提取和匹配应用研究[D].武汉轻工大学,2024.DOI:10.27776/d.cnki.gwhgy.2024.000547.
- [25] 尹源.基于机器学习的企业互联网招聘中简历筛选研究[D].南京邮电大学,2020.DOI:10.27251/d.cnki.gnjdc.2020.000165.
- [26] KARMAKAR P, TENG W S, LU G. Thank you for attention: A survey on attention-based artificial neural networks for automatic speech recognition[J]. Intelligent Systems with Applications, 2024, 23: 200406. DOI:10.1016/J.ISWA.2024.200406.
- [27] KARTHIKEYAN V, SARANYA P, NATCHIYAR M. A lightweight bidirectional GRU–DCNN hybrid framework for end-to-end automatic speech recognition[J]. Microsystem Technologies, 2026, 32(5): 57. DOI:10.1007/S00542-025-05973-3.
- [28] WANG Q, WU B, XU M, et al. HyperSynergyX: Synergistic drug combination prediction via hypergraph modeling and knowledge graph-enhanced retrieval-augmented generation[J]. IEEE Journal of Biomedical and Health Informatics, 2026. DOI:10.1109/JBHI.2026.3673550.
- [29] KYURKCHIEV P, ILIEV A, KYURKCHIEV N. Semantic search for system dynamics models using vector embeddings in a cloud microservices environment[J]. Future Internet, 2026, 18(2): 86. DOI:10.3390/FI18020086.
- [30] 何苗.面向智能招聘的人岗匹配方法研究[D].国防科技大学,2022.DOI:10.27052/d.cnki.gzjgu.2022.000097.